

DEZVOLTAREA UNEI APLICAȚII PENTRU MANAGEMENTUL UNEI SĂLI DE BOX

Candidat: Denis-Ștefan KOCSIS

Coordonator științific: conf.dr.ing. Octavian Stefan

Sesiunea: Iulie 2026

CUPRINS

REZUMAT	4
1 INTRODUCERE	5
2 NOȚIUNI TEORETICE ȘI TEHNOLOGII.....	6
2.1 NOTIUNI GENERALE.....	6
2.1.1 INTRODUCERE APLICĂȚII DESKTOP	6
2.1.2 LIMBAJUL C#	7
2.1.3 LIMBAJUL PYTHON	7
2.1.4 PROTOCOLUL HTTP	8
2.1.5 LIMBAJUL XAML	9
2.1.6 LIMBAJUL JAVASCRIPT	9
2.1.7 LIMBAJUL CSS.....	10
2.1.8 INTELIGENTA ARTIFICIALA IN ANALIZA VIDEO	11
2.2 TEHNOLOGII UTILIZATE.....	12
2.2.1 WPF	13
2.2.2 ENTITY FRAMEWORK.....	14
2.2.3 CSNAKES.....	15
2.2.4 ASP.NET CORE	15
2.2.5 SAM2 ȘI YOLO.....	15
2.2.6 LIVECHARTS.....	17
2.2.7 VISUAL STUDIO 2022.....	17
2.2.8 SQLLITE	18
2.2.9 ROBOFLOW	19
3 ARHITECTURA SI METODOLOGIE DE PROIECTARE.....	20
3.1 ARHITECTURA MVVM.....	20
3.2 STRUCTURA SI ARHITECTURA GENERALA A PROIECTULUI	22
3.3 STRUCTURA BAZEI DE DATE.....	23
3.4 SCHEME SI DIAGrame FUNCTIONALE ALE APLICATIEI.....	24
3.4.1 DIAGRAMA DE ACTIVITATE A SERVERULUI HTTP	24
3.4.2 SCHEMA FUNCTIONALA A APLICATIEI.....	26
3.4.3 DIAGRAMA DE ACTIVITATE A INTIALIZARI MEDIULUI PYTHON.....	28
4 IMPLEMENTARE	30
4.1 IMPLEMENTAREA AI	30
4.1.1 COLECTAREA SI PREGATIREA DATELOR.....	30

4.1.2 ANTRENAREA AI	32
4.1.3 SCRIPTUL PYTHON	34
4.1.4 IMPLEMENTARE IN WPF	38
4.1.5 REZULTATE DUPA ANALIZA	42
4.2 IMPLEMENTAREA APLICATIEI WPF	42
4.2.1 IMPLEMENTARE BAZA DE DATE	43
4.2.2 IMPLEMENTARE LOGARE SI SERVER HTTP.....	45
4.2.3 IMPLEMENTAREA COMENZI DE ANALIZA AI.....	49
4.2.4 ADAUGARE SPORTIV	51
4.2.5 ADAUGARE SPORTIV	53
4.2.6 REINOIRE ABONAMENT	54
4.2.7 SCHIMBARE TEMATICA.....	55
4.2.8 ADAUGARE POZA	56
4.2.9 IMPLEMENTARE VIEWMODEL	56
4.2.10 INTERFATA GRAFICA	61
4.2.10.1 LOGIN.....	61
4.2.10.2 PAGINA PRINCIPALA	61
4.2.10.3 DETALI SPORTIV.....	62
4.2.10.4 PREZENTE.....	63
4.2.10.5 FINANTE.....	64
4.2.10.6 INVENTAR	65
4.3 API.....	66
4.3.1 PAGINAWEB	66
4.3.2 CONTROLER PRINCIPAL.....	67
5 CONCLUZIE.....	68
LISTA FIGURI	70
BIBLIOGRAFIE	72

REZUMAT

Lucrarea prezintă dezvoltarea unei aplicații desktop realizată cu tehnologia WPF (Windows Presentation Foundation) destinată administrării unui club de box. Scopul aplicației este gestionarea elementelor administrative ale salii și urmărirea performanței sportivilor.

Aplicația a fost dezvoltată utilizând limbajul C# în .NET Framework, arhitectura fiind una de tip MVVM (Model-View-ViewModel) pentru o întreținere mai ușoară a codului și pentru a fi mai scalabil. Pentru gestionarea datelor a fost utilizată o bază de date SQLite. Sistemul permite administrarea membrilor, gestionarea abonamentelor, urmărirea prezenței la antrenamente și monitorizarea situației financiare a clubului.

Un element interesant al aplicației îl reprezintă integrarea inteligenței artificiale pentru analiza videourilor din antrenamente și meciuri de box. Prin intermediul acestei analize, utilizatorii pot obține statistici specifice meciului analizat.

De asemenea, sistemul include o componentă ASP.NET Web API care ne oferă comunicarea dintre aplicația principală și sportiv. Prin intermediul acesteia, membrii clubului pot verifica în timp real validitatea abonamentului și informații despre progresul lor la club prin intermediul unei pagini web hostate de API.

Rezultatele obținute demonstrează că aplicația contribuie la digitalizarea activităților clubului de kickboxing, reducând timpul necesar operațiunilor administrative. Soluția propusă poate fi extinsă în viitor prin integrarea unor funcționalități suplimentare și a unor modele de inteligență artificială mai avansate.

1 INTRODUCERE

În ultimii ani, dezvoltarea tehnologiilor software a adus noi metode de organizare și administrare al organizațiilor din diverse domenii. Digitalizarea și menținerea tuturor datelor într-un loc centralizat și ușor de accesat un factor important în eficientizarea administrării resurselor. Cluburile sportive detin un volum considerabil de informații. De aceea și ele au nevoie de soluții software, pentru a le ajuta în administrarea afacerii.

În acest context, dezvoltarea unei aplicații software dedicate administrării unui club de box reprezintă o soluție modernă pentru centralizarea și gestionarea informațiilor.

Unul dintre principalele avantaje ale unei aplicații de management ale unei sali de sport este posibilitatea gestionării centralizate a membrilor. Personalul administrativ poate accesa și modifica informațiile sportivilor într-un mod rapid și eficient. Sistemul oferă administrarea abonamentelor și monitorizarea valabilității acestora. Aceste funcționalități contribuie la reducerea volumului de documente fizice și la îmbunătățirea accesului la informații.

De asemenea, aplicația permite urmărirea prezenței la antrenamente, ceea ce ne arată o imagine clară asupra nivelului de participare al sportivilor. Aceste informații sunt utile antrenorilor deoarece permit identificarea sportivilor activi. Evidența prezenței poate contribui la luarea unor decizii mai bine fundamentate privind organizarea grupelor de antrenament, planificarea activităților sportive și pentru a putea atenționa sportivii de prezență dacă există cazul.

Un alt aspect important al aplicației îl reprezintă gestionarea situației financiare a clubului. Prin intermediul sistemului, administratorii pot monitoriza veniturile și cheltuielile salii venite din vânzarea de echipamente/produse sau din reînnoirea abonamentelor și a membrilor noi. Digitalizarea acestor procese contribuie la o centralizare mult mai eficientă a finanțelor salii și un ajutor în luarea deciziilor economice.

O caracteristică modernă a aplicației este integrarea tehnologiilor de inteligență artificială pentru analiza videourilor și extragerea de informații relevante meciului/sparingului. În ultimii ani, inteligența artificială a devenit un instrument important în domeniul sportului, fiind utilizată pentru analiza performanței. Prin analizarea înregistrărilor video aplicația ajută antrenorii să înțeleagă unde mai are sportivul de lucrat dar și o automatizare a numărării loviturilor.

Importanța dezvoltării unor astfel de soluții software este susținută și de tendința generală de digitalizare a activităților sportive. Tot mai multe organizații aleg să implementeze sisteme informatice pentru optimizarea proceselor interne și pentru îmbunătățirea relației cu membrii. Utilizarea tehnologiilor moderne, precum aplicațiile desktop, serviciile web și inteligența artificială, contribuie la crearea unor sisteme eficiente, scalabile și adaptabile cerințelor actuale.

2 NOȚIUNI TEORETICE ȘI TEHNOLOGII

2.1 NOTIUNI GENERALE

Pentru a putea intelege si crea aplicatii desktop interesante, va trebuii sa cunoastem diferite concepte si tehnologi care au fost folosite in acest proiect. Acest capitol va trece prin elementele de baza cum ar fi , ce este o aplicatie desktop , limbajele de programare C# si Python in cazul de fata ,protocolul HTTP ce a fost folosit in comunicarea dintre aplicatia desktop si API , XAML markup language folosit cu WPF, JavaScript folosit la pagina de web pentru verificarea abonamentului

2.1.1 INTRODUCERE APLICĂȚII DESKTOP

O aplicație desktop este un program software care rulează si este instalata direct pe calculatorul/laptoul personal al utilizatorului . Spre deosebire de aplicațiile web, care depend de accesul prin intermediul unui browser , al unei conexiuni la internet si de infrastructura unui tert pentru a asigura functionalitate, aplicațiile desktop funcționeaza independent și depind de aritectura utilizatorului, resursele hardware fiind gestionate de calculatorul gazda. Ele oferă acces rapid la resursele sistemului si nu necesita conexiune la internet.

Scopul unei aplicații desktop este de a furniza utilizatorilor funcționalități specifice într-un mediu performant și ușor de utilizat. Aceste aplicații sunt utilizate într-o varietate de domenii precum managementul afacerilor, procese educaționale digitalizate, sisteme de informatice, proiectarea asistata de calculator (CAD).

O aplicație desktop este alcătuită din două componente principale. Una dintre ele este interfața cu utilizatorul constituita dintr-un ansamblu de elemente vizuale cum ar fi ferestre ,butoane ,formulare ,meniuri etc. Interfața cu utilizatorul reprezintă partea vizibilă a aplicației și ofera utilizatorului modul de a interactiona cu aplicatia in sine .

Logica aplicației gestionează procesarea datelor și implementarea funcționalităților disponibile. Aceasta se ocupă de validarea informațiilor, gestionarea operațiilor efectuate de utilizatori, comunicarea cu baza de date , generarea rezultatelor necesare toate acestea fiind elemente pe care utilizatorul nu le poate vedea.

Comunicarea dintre interfața și logică este una bidirecțională și asincronă bazându-se pe un mecanism de evenimente (Event-Driven Architecture). Acest lucru înseamnă că în momentul în care un utilizator interacționează cu interfața (de exemplu apăsarea unui buton sau scrierea într-un câmp de text) acest lucru declanșează un eveniment care după este preluat de partea logica. Prin colaborarea dintre interfața utilizator și logica aplicației, sistemul oferă o experiență eficientă și intuitivă, adaptată cerințelor domeniului pentru care a fost dezvoltat.

2.1.2 LIMBAJUL C#

C# este un limbaj de programare orientat pe obiecte dezvoltat de Microsoft, care a câștigat popularitate în ultimele decenii fiind folosit destul de des. Este un limbaj puternic și flexibil, cu o sintaxă similară cu C și C++. C# a fost creat pentru a simplifica dezvoltarea aplicațiilor și pentru a ușura tranziția pentru programatorii familiarizați cu C și C++. C# fiind într-o strânsă legătură cu frameworkul .NET

Limbajul C# a fost introdus în 2002 odată cu lansarea Visual Studio .NET 2002 și a crescut constant de-a lungul timpului, ajungând la versiunea 6.0 în 2015. Numele limbajului se referă la C, un limbaj, altă programare și notația muzicală „Sharp”. , care indică o notă jucată cu o jumătate de treaptă mai sus. O alternativă la C# este limbajul Visual Basic, care este, de asemenea, derivat din platforma .NET. Ambele limbi oferă beneficii similare, iar principala diferență este în sintaxă. Programatorii familiarizați cu C, C++, Java sau JavaScript se vor simți confortabil folosind sintaxa C#, în timp ce cei care nu cunosc aceste limbi sau nu sunt familiarizați cu versiunile Visual Basic Oamenii în vârstă pot găsi .NET mai ușor să învețe limbajul Visual Basic.

C# este un limbaj de programare modern și puternic. Ne oferă o abundență de funcții care permit programatorului să creeze diverse aplicații, cum ar fi aplicații desktop, web, mobile și cloud. C# are o sintaxă ușor de înțeles care alături de o bibliotecă extinsă de clase și cadre permit dezvoltarea rapidă și eficientă a aplicațiilor. Limbajul este folosit în diverse domenii, cum ar fi dezvoltarea de jocuri și aplicații de întreprindere cu platforma Unity. C# continuă să fie popular și preferat în comunitatea de programare, datorită suportului activ și resurselor oferite de Microsoft. Este un instrument puternic și inovator pentru construirea de aplicații sofisticate.

În acest proiect C# a fost limbajul principal fiind folosit în crearea aplicației desktop principale care a fost implementată folosind tehnologia WPF (Windows Presentation Foundation) ,C# stă la baza părții logice a aplicației și în același timp a fost folosit și pentru dezvoltarea API-ului unde a fost folosit la programarea controlerului principal. [1]

2.1.3 LIMBAJUL PYTHON

Python este un limbaj de programare de nivel înalt fiind un limbaj cu scop general(limbaj pentru crearea de software într-o varietate cât mai mare de domenii), interpretat, creat de Guido van Rossum în 1991. Acesta a fost proiectat având la bază principiul simplității ceea ce îl face unul dintre cele mai accesibile și ușor de învățat limbaje pentru începători, dar el în același timp poate fi folosit și pentru aplicații complexe.

Unul dintre principalele obiective ale limbajului Python este de a face codul lizibil și ușor de scris, folosind o sintaxă simplă și ușor de înțeles, apropiată de limbajul natural ceea ce duce la creșterea productivității. Din acest motiv, Python este adesea recomandat ca primul limbaj de programare, fiind utilizat pe scară largă în mediul educațional în universități și licee, dar și în industria software unde este folosit în aplicații web, automatizării, inteligența artificială. El fiind comparat cu limbaje precum Java, Perl și Javascript.

De-a lungul timpului, Python a evoluat constant, devenind un limbaj extrem de popular în domenii precum dezvoltarea web, știința datelor, inteligența artificială, automatizări, machine learning și scripting. Această popularitate este susținută de o comunitate vastă și activă, precum și de o colecție impresionantă de biblioteci și framework-uri, cum ar fi Django și Flask pentru dezvoltare web sau NumPy, Pandas și TensorFlow pentru analiza datelor și inteligență artificială.

Python este un limbaj multi-paradigmă, suportând programarea orientată pe obiecte, programarea funcțională și programarea procedurală și în același timp are o gamă extensă de librării built-in. Această flexibilitate îl face potrivit pentru o gamă largă de aplicații, de la proiecte simple până la sisteme complexe utilizate în mediul profesional.

În cadrul acestui proiect, Python este utilizat pentru analiza videourilor folosind inteligența artificială. Pentru logica de detecție a mișcărilor de luptă este utilizat un script Python care afișează rezultatele predicțiilor oferite de modelul AI. [2]

2.1.4 PROTOCOLUL HTTP

Hypertext Transfer Protocol (HTTP) este protocolul pentru operarea și comunicarea pe Internet, el fiind protocolul de bază folosit de browserele web atunci când dorim să accesăm diferite site-uri sau resurse de pe internet, asigurând comunicarea dintre client și server. HTTP funcționează într-un model client/server, clientul solicită resurse de la server, iar serverul răspunde cu acele resurse. Comunicarea se face prin cereri și răspunsuri HTTP. Portul comun unde se afla un serviciu HTTP este portul 80.

O solicitare HTTP este inițiată de client și conține informații precum metoda (GET, POST, PUT, DELETE), adresa URL a resursei și câmpuri suplimentare, cum ar fi anteturile și corpul cererii. Serverul primește cererea și returnează un cod de stare care indică succesul sau eșecul cererii și conținutul solicitat sau informații suplimentare.

Un aspect important al HTTP este că este un protocol fără stare. Aceasta înseamnă că serverul nu stochează nicio informație despre solicitările anterioare ale clientului. Acest lucru face arhitectura web mai extensibilă și mai flexibilă.

Protocolul HTTP nu este securizat el transmițând datele în plain text ceea ce poate fi un risc de securitate de aceea a fost inventat protocolul HTTPS care este identic cu HTTP dar acesta criptează datele înainte ca acestea să fie trimise.

O altă caracteristică a HTTP este suportul pentru hipertext. Acest lucru vă permite să creați legături între diferite resurse web. Aceste link-uri sunt esențiale pentru navigarea pe web și link-uri către pagini și site-uri web. HTTP a evoluat de-a lungul timpului de la versiunea sa originală HTTP/1.0 la HTTP/1.1 la HTTP/2, rezultând îmbunătățiri semnificative ale performanței și eficienței transferului de date.

Pe scurt, HTTP (Hypertext Transfer Protocol) este un protocol esențial pentru comunicarea cu diferite site-uri pe internet. Transferă informații între server și client pentru a asigura funcționarea și accesibilitatea site-ului și oferă erori sau atenționări dacă ceva nu a funcționat în mod corespunzător. Evoluția continuă a HTTP și-a îmbunătățit considerabil performanța și capacitățile.

În acest proiect protocolul HTTP a fost folosit în comunicarea dintre API și aplicația desktop și pentru pagina web aferentă prezentei sportivilor. Prin intermediul acestui protocol API (Aplicația Client) transmite printr-un POST request numele dat de sportiv. [3]

2.1.5 LIMBAJUL XAML

Limbajul XAML (Extensible Application Markup Language) este un limbaj de marcare pentru interfața cu utilizatorul, dezvoltat de Microsoft în anul 2006. El este folosit pentru definirea interfețelor grafice ale aplicațiilor construite în .NET. Acesta permite descrierea elementelor vizuale și a structurii unei aplicații într-un mod declarativ, separând partea de prezentare de logica aplicației. XAML este utilizat în tehnologii precum WPF (Windows Presentation Foundation), UWP și WinUI, fiind unul dintre cele mai importante instrumente pentru dezvoltarea aplicațiilor desktop moderne.

Limbajul XAML a fost introdus odată cu lansarea tehnologiei WPF și a devenit rapid o componentă esențială a tehnologiilor ecosistemului .NET. Bazat pe sintaxa XML, XAML oferă dezvoltatorilor posibilitatea de a defini ferestre, controale, stiluri, animații și alte elemente grafice într-un mod clar și organizat și în același timp este foarte ușor de citit. Prin utilizarea sa, interfața aplicației poate fi proiectată și modificată independent de partea logică prin folosirea data bindingului. Acest lucru permite programatorului să nu amestece partea logică cu partea vizuală, ajutând la o mentinere mult mai curată a codului și o scalabilitate mai ușoară.

Un avantaj important al limbajului XAML este separarea clară dintre interfața utilizatorului și logica aplicației. Această abordare facilitează mentenanța codului, permite colaborarea eficientă dintre dezvoltatori și designeri și contribuie la implementarea unor arhitecturi moderne precum MVVM (Model-View-ViewModel). De asemenea, XAML oferă suport pentru data binding, stiluri personalizate, șabloane de control și animații, contribuind la crearea unor aplicații cu interfețe atractive și ușor de utilizat.

În cadrul acestui proiect, XAML a fost utilizat pentru proiectarea și implementarea interfeței grafice a aplicației desktop. Prin intermediul acestuia au fost definite ferestrele aplicației, controalele vizuale, structura paginilor și legăturile dintre interfață și date. Utilizarea XAML a permis realizarea unei interfețe moderne și intuitive, menținând în același timp o separare clară între partea vizuală și logica folosindu-se de arhitectura MVVM și proprietatea de data binding a XAML-ului. [4]

2.1.6 LIMBAJUL JAVASCRIPT

JavaScript (JS) este un limbaj de programare interpretat folosit în principal pentru dezvoltarea de aplicații web interactive și dinamice. Deși la început JavaScript a fost văzut ca un limbaj simplu pentru scripturi mici, el a evoluat în unul dintre pilonii dezvoltării aplicațiilor web. Acesta rulează în browserul web al unui utilizator și îi permite acestuia să manipuleze și să interacționeze cu elementele paginii web.

JavaScript oferă caracteristici puternice, cum ar fi manipularea DOM (Document Object Model), care poate fi folosită pentru a schimba și actualiza în mod dinamic elementele unei pagini web. Aceasta înseamnă că elementele pot fi create, modificate sau șterse fără a reîncărca întreaga pagină.

Un alt aspect important al JavaScript este arhitectura bazată pe evenimente. Acest lucru face posibil să reacționați la acțiunile utilizatorului, cum ar fi clic pe butoane, trecerea cursorului peste elemente, un scroll, apăsarea unei taste sau simpla mișcare a mouse-ului. Limbajul capturează aceste acțiuni și declanșează funcții specifice. Puteți utiliza evenimente și funcții conexe pentru a implementa acțiuni și comportamente interactive pe paginile dvs. web.

JavaScript este un limbaj versatil care permite dezvoltarea de aplicații complexe. Manipularea structurilor de date, funcțiile, condițiile și buclele sunt suportate, facilitând dezvoltarea algoritmilor și a logicii programului.

Un alt motiv pentru care JavaScript a ajuns la popularitatea de astăzi a fost crearea Node.js în 2009. Odată cu apariția Node.js dezvoltatorii software/web au putut folosi JavaScript și pe partea de server (backend). Acest lucru înseamnă că o echipă de programatori poate scrie o întreaga aplicație web într-un singur limbaj de programare.

JavaScript are, de asemenea, o comunitate mare și activă care a dezvoltat multe biblioteci și cadre populare, cum ar fi jQuery, React.js și Angular.js. Acestea oferă caracteristici și abstracții suplimentare care simplifică dezvoltarea și întreținerea codului JavaScript. JavaScript este folosit și în afara browserului web atunci când se dezvoltă aplicații mobile și server prin platforme precum Node.js.

Pe scurt, JavaScript este limbajul de programare esențial pentru dezvoltarea aplicațiilor web interactive. Acest lucru vă permite să interacționați cu conținutul site-ului web, să gestionați evenimente și să creați interactivitate. Versatilitatea JavaScript și suportul activ al comunității îl fac un instrument puternic în dezvoltarea de aplicații web moderne.

În acest proiect JavaScript a fost folosit în partea de client web pentru a trimite o cerere de tip HTTP POST cu numele sportivului către componenta server aplicația desktop WPF și pentru a oferi fiecărui sportiv un ID personal, ca aceștia să nu poată să introducă mai multe nume după ce primul nume introdus a fost identificat ca un nume valid ce nu are abonamentul expirat. [5]

2.1.7 LIMBAJUL CSS

CSS, prescurtare de la Cascading Style Sheets, este un limbaj utilizat pentru stilizarea paginilor web și a interfețelor grafice dezvoltate cu ajutorul tehnologiilor web, precum HTML sau XML. Acest limbaj nu este un limbaj de programare propriu-zis, deoarece nu conține structuri logice complexe precum condiții sau bucle. Acesta are un rol pur descriptiv, care stabilește cum sunt prezentate designul și elementele, utilizatorului.

Limbajul CSS a fost propus pentru prima dată în anul 1994 de Håkon Wium Lie, iar prima versiune oficială, CSS1, a fost publicată de World Wide Web Consortium în anul 1996. Scopul principal a fost de a separa conținutul unei pagini web de partea sa vizuală, facilitând astfel modificarea aspectului unei aplicații fără a schimba structura de bază a documentului.

HTML. Astfel această abordare revoluționară a permis dezvoltatorilor să poată realiza modificări asupra designului, culorilor, poziționării elementelor sau dimensiunilor fără a modifica structura de bază a fișierului HTML.

CSS permite controlarea mai multor aspecte vizuale, precum: culorile, fonturile, dimensiunile textului, poziționarea elementelor, spațierea, fundalurile, marginile, animațiile și adaptarea interfeței pentru diferite dimensiuni ale ecranului. Dincolo de partea estetică, acesta are un rol foarte important în dezvoltarea interfețelor moderne, deoarece contribuie la îmbunătățirea experienței utilizatorului și la crearea unui design mai plăcut, mai ordonat și mai ușor de utilizat.

Unul dintre marile avantaje ale limbajului CSS este faptul că acesta permite reutilizarea stilurilor în mai multe pagini sau componente ale unei aplicații. Prin utilizarea unui fișier CSS separat, aspectul unei pagini poate fi administrat mai eficient, iar modificările pot fi realizate rapid, eliminând necesitatea schimbării fiecărui element în parte. Acest lucru facilitează dezvoltarea și întreținerea aplicațiilor web, mai ales în cazul proiectelor care conțin mai multe pagini sau componente vizuale.

În această lucrare de licență CSS a fost utilizat pentru stilizarea paginii de la API, având rolul de a organiza și îmbunătăți aspectul elementelor afișate. Prin intermediul acestuia au fost definite caracteristici vizuale precum poziționarea componentelor, culorile, dimensiunile, spațierea și modul de afișare al informațiilor, contribuind astfel la realizarea unei interfețe plăcute care să faciliteze utilizarea. [6]

2.1.8 INTELIGENTA ARTIFICIALA IN ANALIZA VIDEO

Inteligența Artificială (AI) reprezintă un domeniu al informaticii moderne care se ocupă cu dezvoltarea sistemelor capabile să simuleze anumite procese asociate inteligenței umane, precum învățarea continuă, recunoașterea modelelor și tiparelor, analiza informațiilor și luarea deciziilor algoritmice. În ultimii ani, progresul tehnologic și creșterea puterii de calcul au contribuit la dezvoltarea unor soluții avansate bazate pe inteligență artificială, trecând de limitele teoretice, ajungându-se la o varietate largă de soluții bazate pe inteligența artificială.

Unul dintre domeniile în care inteligența artificială a avansat enorm și și-a demonstrat utilitatea este analiza și procesarea conținutului video. Materialele video conțin o cantitate foarte mare de informații vizuale, iar procesarea manuală a acestora poate fi dificilă și costisitoare din punct de vedere al timpului pierdut și al erorilor umane. Prin utilizarea algoritmilor de inteligență artificială, procesul de analiză poate fi automatizat, permițând identificarea rapidă a obiectelor, persoanelor, acțiunilor sau evenimentelor prezente într-o secvență video.

Scopul utilizării inteligenței artificiale în analiza video este extragerea automată a informațiilor relevante din fluxurile video și transformarea acestora în date utile pentru utilizator. Sistemele moderne pot recunoaște diferite obiecte, urmări mișcarea acestora și identifica ID-ul acestora pe parcursul a mai multor frame-uri, recunoaște anumite activități și genera informații statistice pe baza datelor colectate, minimizând necesitatea

supravegherii umane continue. Aceste funcționalități contribuie la creșterea eficienței și la reducerea necesității intervenției umane în procesul de monitorizare și analiză.

Procesul de analiză video bazat pe inteligență artificială implică preluarea cadrelor video, procesarea acestora și identificarea caracteristicilor relevante. Algoritmii specializați analizează fiecare cadru și extrag informații despre elementele prezente în imagine. Rezultatele obținute sunt apoi interpretate și utilizate pentru realizarea diferitelor funcționalități ale aplicației.

Tehnologiile de analiză video bazate pe inteligență artificială sunt utilizate în numeroase domenii, precum supravegherea și securitatea, managementul traficului, industria sportivă, medicina, automatizarea proceselor industriale și vehiculele autonome, și devin din constant mai bune la ceea ce fac. Capacitatea acestor sisteme de a procesa rapid volume mari de date vizuale le transformă într-un instrument important pentru luarea deciziilor și optimizarea activităților desfășurate în diverse medii. Totuși un dezavantaj al acestei tehnologii este necesitatea unui hardware performant deoarece antrenarea unui AI și analizarea unui video necesită multă putere computațională.

Prin integrarea inteligenței artificiale în aplicațiile de analiză video se obține un sistem capabil să interpreteze automat informațiile vizuale și să furnizeze rezultate într-un timp redus. Acest lucru contribuie la creșterea preciziei procesului de analiză, la reducerea costurilor operaționale și la îmbunătățirea eficienței generale a sistemelor informatice care utilizează date video. [7]

2.2 TEHNOLOGII UTILIZATE

În cadrul lucrării am folosit o gamă largă de tehnologii software moderne pentru dezvoltarea aplicațiilor desktop, aplicațiilor web și inteligenței artificiale ale sistemului.

Printre aceste tehnologii se regăsește WPF frameworkul principal în dezvoltarea aplicației desktop, ea este o componentă a ecosistemului .NET. Acesta oferă suportul pentru realizarea unor aplicații desktop moderne cu interfețe interactive, iar împreună cu arhitectura MVVM permite separarea logicii aplicației de partea vizuală.

Pentru dezvoltarea părții web, am folosit ASP.NET pentru a crea interfața web și comunicarea cu aplicația desktop.

IDE-ul folosit a fost Visual Studio 2022 care este într-o strânsă legătură cu ecosistemul .NET. Acesta oferă unelte utile în scrierea, testarea și depanarea codului, contribuind la creșterea productivității.

Pentru integrarea AI cu Python în .NET, afisarea graficelor, interacțiunea cu baza de date s-au folosit mai multe biblioteci care vor fi explicate în detaliu în subcapitolele următoare.

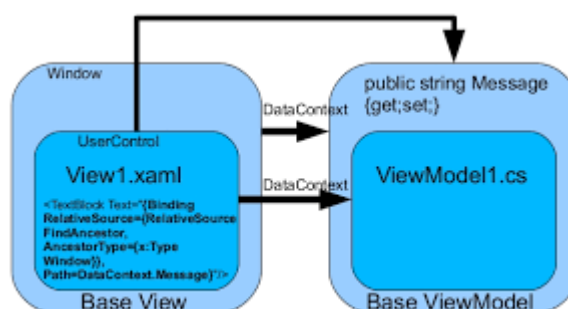
AI-urile folosite au fost YOLO5 și SAM2 ambele două modele open-source recunoscute în domeniul procesării de imagini și al analizei video, ele au fost folosite în analiza video, pentru a extrage informații cum ar fi, în ce frame este detectată o lovitură, diferențierea dintre cei doi luptători și masca fiecărui luptător pe parcursul fluxului video.

2.2.1 WPF

WPF este un framework dezvoltat de Microsoft pentru crearea aplicațiilor desktop moderne. WPF este succesorul modern al clasicului Windows Forms. Acesta a fost introdus în anul 2006 ca parte a platformei .NET Framework și încă rămâne o soluție solidă în dezvoltarea de aplicații desktop și în zilele de azi.

Spre deosebire de tehnologiile mai vechi pentru dezvoltarea aplicațiilor desktop, precum Windows Forms, WPF utilizează un sistem de randare bazat pe DirectX. Acest lucru schimbă complet modul în care partea vizuală interacționează cu hardware-ul calculatorului, interfața nu mai este dependentă de procesor ci acum este accelerată de placa video, ceea ce permite realizarea unor interfețe grafice mai performante și mai rapide. Acest lucru face posibilă integrarea facilă și mult mai rapidă a animațiilor, efectelor grafice, elementelor multimedia și a controalelor personalizate, contribuind la dezvoltarea unor aplicații moderne și intuitive.

Una dintre caracteristicile principale ale frameworkului WPF este utilizarea limbajului XAML. Cu ajutorul binding-urilor XAML funcționează mână în mână cu codul din spate dar în același timp sunt complet separate ceea ce înseamnă că schimbări în partea vizuală nu vor afecta partea logică și viceversa. Prin această abordare, dezvoltatorii pot organiza mai eficient codul sursă și pot modifica aspectul aplicației fără a afecta funcționalitatea acesteia. În figura de mai jos se poate observa cum un element din XAML este bind-uit la o proprietate din clasa C#. [8]



Figură 1 MVVM

WPF oferă suport pentru numeroase funcționalități avansate, printre care se numără data binding-ul, stilurile personalizate, șabloanele de control (Control Templates), șabloanele de date (Data Templates) și sistemul de resurse și animațiile StoryBoards.

Un alt avantaj important al WPF este integrarea sa excelentă cu arhitectura MVVM (Model-View-ViewModel). Această arhitectură este utilizată frecvent în aplicațiile moderne deoarece separă clar responsabilitățile dintre modelul de date, interfața utilizatorului și logica de prezentare, acest lucru face codul mult mai modular și ușor de întreținut.

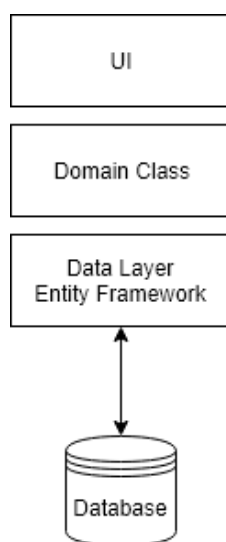
Frameworkul WPF pune la dispoziția dezvoltatorilor o gamă largă de controale grafice predefinite, precum butoane, liste, tabele, meniuri, ferestre de dialog și controale pentru introducerea datelor. Acestea pot fi personalizate cu ușurință prin utilizarea stilurilor și a șabloanelor, oferind posibilitatea dezvoltării unor interfețe adaptate cerințelor fiecărui

proiect. De asemenea, WPF suportă dezvoltarea interfețelor responsive, capabile să se adapteze automat la diferite rezoluții și dimensiuni ale ferestrelor.

În cadrul acestui proiect, WPF a fost utilizat pentru dezvoltarea aplicației desktop și este partea centrală a proiectului restul componentelor învârtindu-se în jurul său. Prin intermediul acestuia au fost implementate toate ferestrele aplicației, controalele grafice necesare administrării datelor și funcționalitățile de interacțiune cu utilizatorul. Utilizarea WPF a permis realizarea unei interfețe moderne și intuitive, capabile să ofere acces rapid la informații relevante și să faciliteze procesul de gestionare a sportivilor.

2.2.2 ENTITY FRAMEWORK

Entity Framework este o tehnologie open-source dezvoltată de Microsoft în anul 2008, este un object-relational mapper pentru ecosistemul .NET. Acest Framework ajută programatorii să lucreze cu bazele de date fără a trebui să se implice în scrierea de cod SQL și tehnicilor de acces pentru lucrul cu baza de date. În figura de mai jos se poate vedea legătura dintre Entity Framework, baza de date și clasa în care vor fi convertite tabelele și rândurile din baza de date.



Figură 2 Entity

Această abordare elimină o mare parte din codul repetitiv pe care programatorii trebuiau să îl scrie în trecut folosind tehnologii mai vechi, cum ar fi ADO.NET. În loc să deschidem manual conexiuni către baza de date, să creăm comenzi și să citim rezultatele linie cu linie, Entity Framework preia toate aceste sarcini repetitive în fundal. El acționează ca un intermediar inteligent care traduce limbajul programării orientate pe obiecte în limbajul bazelor de date relaționale, gestionând automat conexiunile, mapările și interogările necesare pentru rularea aplicației.

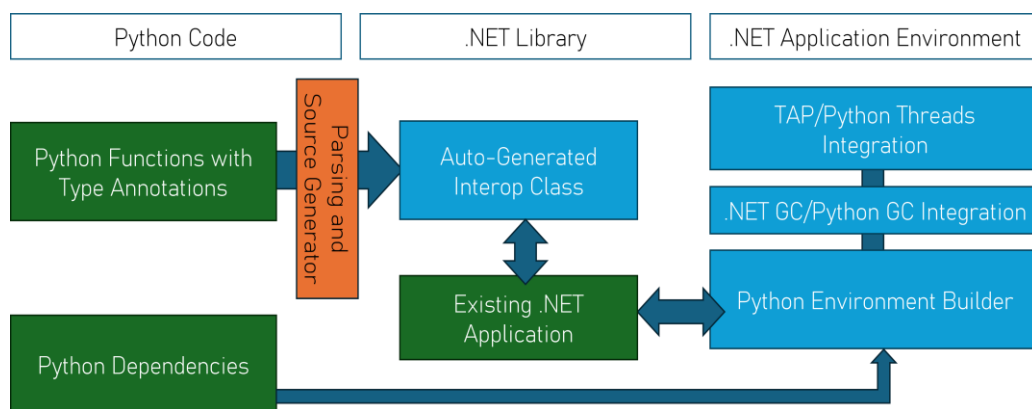
El abstracționează lucrul cu rândurile și coloanele din bazele de date relaționale ele fiind integrate ca și clase C#. Datele din baza de date putând fi accesate și modificate folosind query-uri LINQ în loc de cele SQL.

Poate salva schimbări în baza de date care au fost cauzate de diferite operațiuni cum ar fi inserări, deleții și actualizări. [9]

2.2.3 CSNAKES

CSnakes este o unealtă făcută pentru ecosistemul .NET care permite incorporarea codului de Python în proiectele .NET fără a avea nevoie de REST, HTTP sau servicii suplimentare.

Cu CSnakes programatorul poate crea un mediu virtual închis unde codul Python și toate dependențele necesare pot fi rulate iar după aceea ele sunt învelite în clase C# care apoi pot fi folosite în aplicație. În figura de mai jos se poate vedea cum se întâmplă acest lucru [10]



Figură 3 CSnakes

2.2.4 ASP.NET CORE

ASP.NET Core este un framework open-source disponibil pe mai multe platforme, care oferă performanțe ridicate, dedicat dezvoltării de aplicații web moderne utilizând platforma .NET. Mulțumită caracteristicilor pe care acest framework le oferă, precum performanța, faptul că unește backendul (MVC și Web API) într-un singur model, dar și faptul că este pregătit pentru cloud, proiectat special pentru a rula eficient în containere (Docker) și servicii cloud (Azure, AWS) îl face o alegere populară în rândul dezvoltatorilor. [11]

2.2.5 SAM2 ȘI YOLO

SAM2 (Segment Anything 2) este un model de segmentare lansat de Meta AI care permite utilizatorului să seteze coordonate într-o imagine și să genereze o mască de segmentare pentru punctele respective. De asemenea, această mască rămâne persistentă în fiecare cadru dintr-o filmare, urmărind astfel obiectele dorite din aceasta.

SAM 2 este un model preantrenat pe un set de 51.000 de videoclipuri și 643.000 de măști de segmentare. De asemenea, există patru versiuni ale acestuia: tiny, small, base, large care permit dezvoltatorilor să aleagă în funcție de nevoile specifice aplicației

dezvoltate: cu cât modelul este mai mare, cu atât este mai precis, dar și performanțele au de suferit.

În cadrul acestei lucrări a fost ales modelul large, deoarece nu au fost impuse limitări de performanță, iar o acuratețe mai ridicată a fost un lucru dorit. Din punct de vedere al funcționalității a fost utilizat în a crea măști pentru luptătorii dintr-un video și a îi diferenția. În imaginea de mai jos putem vedea cum arata o segmentare realizata de SAM 2 pe diferite masini, pe o autostrada. [12]



Figură 4 SAM

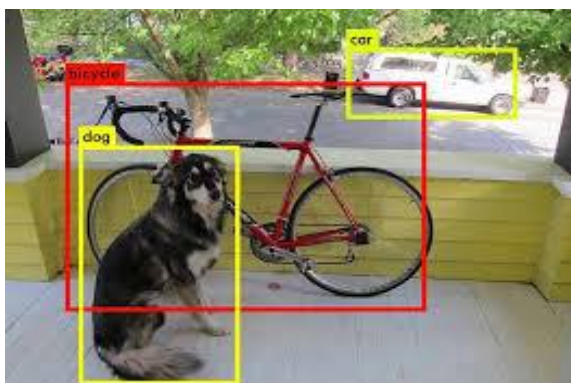
YOLO(You Only Look Once) reprezintă o serie de sisteme de detecție a obiectelor în timp real bazat pe o rețea neuronală convoluțională. Acesta a fost introdus prima dată în anul 2015, iar de atunci a trecut prin mai multe îmbunătățiri, devenind astfel unul dintre cele mai populare frameworkuri de detecție a obiectelor.

Principiul de funcționare al YOLO se bazează pe analizarea imaginii într-o singură etapă, spre deosebire de alte metode de detecție care parcurg mai mulți pași pentru identificarea obiectelor. Acest lucru permite modelului să detecteze rapid obiectele dintr-o imagine, oferind atât clasa obiectului identificat, cât și poziția acestuia prin intermediul unor dreptunghiuri de delimitare, numite bounding boxes.

Un avantaj important al acestui framework este viteza ridicată de procesare, motiv pentru care YOLO este foarte utilizat în aplicații care necesită detecție în timp real, precum supravegherea video, sistemele autonome, robotica sau aplicațiile de recunoaștere vizuală. De asemenea, versiunile mai noi ale modelului au adus îmbunătățiri în ceea ce privește acuratețea, viteza și capacitatea de a detecta obiecte de dimensiuni diferite.

În cadrul acestei lucrări, YOLO a fost utilizat pentru detecția luptătorilor din imagini, având rolul de a identifica rapid elementele de interes și de a oferi informații despre poziția acestora.

În figura de mai jos putem observa outputul dat de YOLO când a primit o imagine oarecare și a reușit să detecteze diferitele obiecte aflate în ea cum ar fi câinele, bicicleta și mașina. [13]



Figură 5 YOLO

2.2.6 LIVECHARTS

LiveCharts este o librărie open-source pentru .NET care ne ofera mai multe optiuni moderne si dinamice de integrare a graficelor,diagramelor si altor elemente de vizualizare a datelor.

Un avantaj al acestei biblioteci este actualizarea datelor in timp real ,graficile schimbându-se in timp real odata cu modificarea datelor.Ea poate fi integrara direct in codul XAML ,oferind suport specific ,precum data binding ,stilizaare si arhitectura MVVM.

Biblioteca pune la dispozitie numeroase tipuri de grafice cum ar fi , grafice circulare, grafice cu bare, histograme si multe altele . In figura de mai jos putem vedea diferite tipuri de grafice oferite de LiveCharts [14].



Figură 6 LIVECHARTS

2.2.7 VISUAL STUDIO 2022

Visual Studio este un IDE dezvoltat de Microsoft, care oferă un set complet de unelte pentru dezvoltarea de aplicații, precum ASP.NET, aplicații desktop, aplicații mobile, etc.

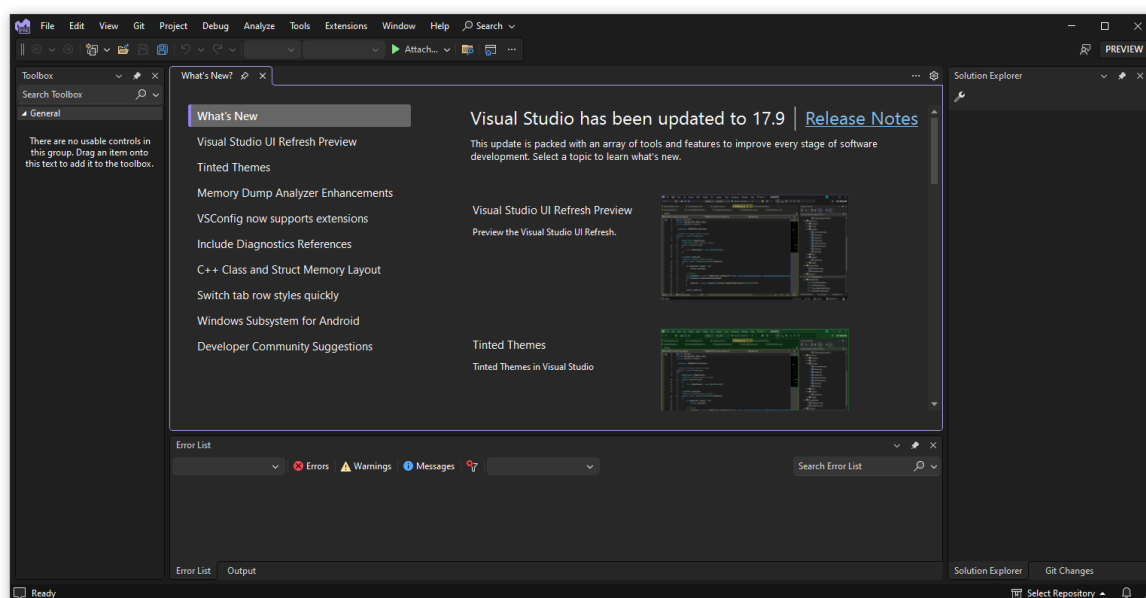
Acesta este echipat cu un editor de cod care dispune de IntelliSense și aranjare automată a codului, aceste funcții facilitând dezvoltarea de cod. IntelliSense ajută programatorul prin sugestii automate de completare, afișarea parametrilor și evidențierea erorilor încă din timpul scrierii codului, reducând erorile banale de scriere și optimizând timpul în care un dezvoltator creează o soluție software.

De asemenea, acesta este echipat cu un debugger, dar și cu alte unelte specifice pentru design de interfețe, design de pagini web, design de clase și baze de date. Debuggerul permite identificarea și corectarea erorilor din aplicație, oferind posibilitatea de

a urmări execuția codului pas cu pas. Acest lucru este foarte important în dezvoltarea aplicațiilor, deoarece pentru realizarea unei aplicații care să ofere o experiență bună utilizatorului este necesară testarea software.

Acest IDE oferă o gamă largă de limbaje de programare suportate, precum C, C++, Visual Basic, .NET, C#, etc.. Datorită acestor caracteristici, Visual Studio este utilizat frecvent în dezvoltarea aplicațiilor bazate pe platforma .NET, dar și în proiecte complexe care necesită mai multe tehnologii.

În cadrul acestei lucrări de licență, Visual Studio a fost utilizat pentru dezvoltarea și administrarea codului aplicației, oferind un mediu complet pentru scrierea, testarea și depanarea componentelor software. Prin intermediul acestui IDE, procesul de dezvoltare a fost mai ușor de gestionat, iar integrarea tehnologiilor utilizate în proiect a fost realizată într-un mod mai eficient. [15]



Figură 7 VISUAL STUDIO

2.2.8 SQLITE

SQLite este o baza de data relationala creata de D. Richard Hipp in anul 2000 , ea a fost scrisa in limbajul C si nu este o aplicatie in sine ci ea este o librarie care este integrata in proiectele software.

Baza de date nu are un proces pe care ruleaza,precum MySQL sau PostgreSQL, ea este accesata printr-un linker care integreaza biblioteca in program .SQLite stocheaza toata baza de date intr-un singur fisier ceea ce reduce complexitatea accesari bazei de date ,deoarece nu este necesara configurarea unui server de baze de date si in acelasi timp face baza de date foarte portabila putand fi copiată si mutată foarte usor.

Datorita consumului redus de resurse si dimensiunii mici , SQLite este o alegere excelente pentru aplicatiile de tip embedded, aplicatii mobile, software desktop si dispozitive IOT [16].

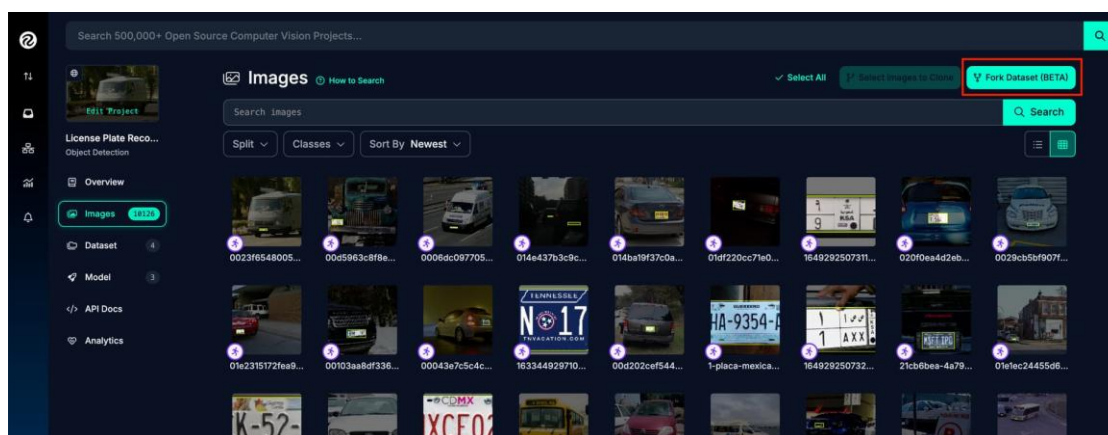
2.2.9 ROBOFLOW

Roboflow este o platforma cloud de dezvoltare care facilitează procesul de creare antrenare și implementare a modelelor de învățare automată pentru viziune computerizată. Aceasta oferă unelte pentru încărcarea, organizarea, etichetarea și preprocesarea imaginilor, reducând astfel timpul necesar pregătirii unui set de date.

Acesta a fost lansat în anul 2020 și oferă dezvoltatorilor un mediu perfect pentru dezvoltarea de modele de inteligență artificială bazată pe imagini, prin intermediul platformei fiind puse la dispoziție atât dataseturi cât și opțiunea de a antrena modele prin cloud. De asemenea, platforma permite exportarea dataseturilor în formate compatibile cu mai multe frameworkuri de detecție și clasificare a obiectelor.

În cadrul acestei lucrări Roboflow a fost utilizat pentru obținerea de date(imagini) pentru antrenarea modelului de inteligență artificială de detecție a mișcărilor luptătorilor.

În figura 8 de mai jos este prezentată o captură de ecran a interfeței roboflow [17].



Figură 8 ROBOFLOW

3 ARHITECTURA SI METODOLOGIE DE PROIECTARE

3.1 ARHITECTURA MVVM

Pentru a pastra o separare clara dintre interfata , partea logica si datele utilizate de sistem si pentru a putea face procesul de dezvoltare mai usor de intretinut, testat, si extins, am folosit arhitectura de tip MVVM (Model-View-ViewModel). Aceasta arhitectura imparte aplicatia in 3 componente principale.

Partea de Model se ocupa cu reprezentarea datelor si logica de business aici apar fisiere care definesc tabele bazei de date , clasa ce preia rezultatul JSON , clasa ce se ocupa de conectarea si crearea bazei de date si clasa ce porneste serverul de http intern al aplicatiei.

Partea de View este reprezentata de interfața grafică ea fiind formata din pagini XAML. Ferestrele aplicației conțin elemente precum liste de sportivi, câmpuri pentru introducerea datelor, butoane pentru adăugare sau ștergere și grafice pentru afișarea statisticilor. View-ul nu conține logică de business, rolul său fiind doar de prezentare și interacțiune vizuală.

Partea de ViewModel a avut rolul central in aplicatia WPF el fiind in stransa legatura si cu partea de View si cu partea de Model . De exemplu, atunci când utilizatorul apasă un buton pentru adăugarea unui sportiv, comanda asociată este preluată de ViewModel, care validează datele și apelează metodele necesare din Model pentru comunicarea cu baza de date si salvarea informațiilor.

Legatura dintre View si ViewModel este realizata cu ajutorul unor mecanisme oferite de WPF special create pentru implementarea MVVM.

Unul dintre acestea este Data Binding-ul, care face legatura dintre proprietatile unei clase ViewModel si elemente cum ar fie textbox-uri , checkbox-uri si alte elemente ale interfetei .

Un al doilea element este interfata INotifyPropertyChanged care notifica View-ul atunci cand valoare unei proprietati din ViewModel se schimba ca sa o poata actualiza in timp real. Toate clasele din ViewModel mostenesc din clasa principala ViewModelBase implementarea acestei interfete. In figura de mai jos apare clasa ViewModelBase. [18]

```
1 using System;
2 using System.Collections.Generic;
3 using System.ComponentModel;
4 using System.Linq;
5 using System.Text;
6 using System.Threading.Tasks;
7
8 namespace Modern.ViewModel
9 {
10     20 references
11     public class ViewModelBase : INotifyPropertyChanged
12     {
13         public event PropertyChangedEventHandler? PropertyChanged;
14
15         99+ references
16         protected void OnPropertyChanged(string propName)
17         {
18             PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propName));
19         }
20     }
21 }
```

Figură 9 CLASA VIEWMODELBASE

Un al treilea element este interfata ICommand. Aceasta permite separarea evenimentelor generate de interfața grafică de logica aplicației. În loc ca un buton să execute direct cod în fișierul XAML sau în code-behind, acțiunea este asociată unei comenzi definite în ViewModel. În cadrul aplicației dezvoltate, comenzile au fost utilizate pentru operații precum adăugarea, modificarea și ștergerea sportivilor, actualizarea informațiilor afișate în interfață, încărcarea datelor din baza de date și navigarea între diferitele pagini ale aplicației. Toate clasele de tip Command mostenesesc clasa CommandBase care implementeaza aceasta interfata. urmatoarea imagine arata clasa CommandBase.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Input;

namespace Modern.ViewModel
{
    34 references
    public abstract class CommandBase : ICommand
    {
        public event EventHandler? CanExecuteChanged;

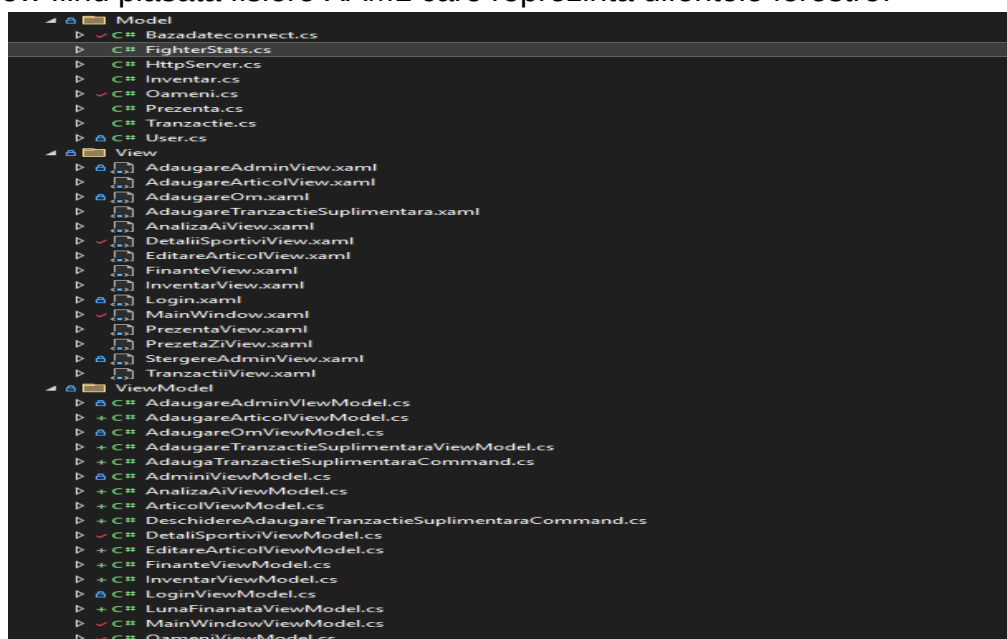
        0 references
        public virtual bool CanExecute(object? parameter)
        {
            return true;
        }

        34 references
        public abstract void Execute(object parameter);

        0 references
        protected void OnCanExecuteChanged()
        {
            CanExecuteChanged?.Invoke(this, EventArgs.Empty);
        }
    }
}
```

Figură 10 CLASA COMMANDBASE

În figura de mai jos este reprezentată structura fișierelor după modelul MVVM. În folderul de View fiind plasate fișiere XAML care reprezintă diferitele ferestre.



Figură 11 STRUCTURA FISIERE

3.2 STRUCTURA SI ARHITECTURA GENERALA A PROIECTULUI

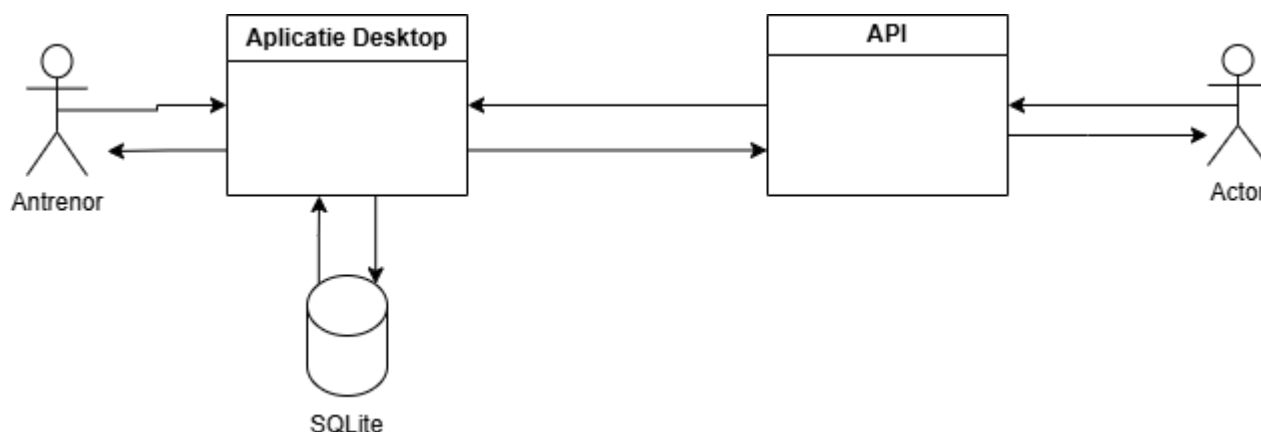
Proiectul este format din doua componente principale care comunica intre ele pentru a gestiona datele primite de utilizator .

Prima componenta este aplicatia WPF ea fiind componenta principala a proiectului unde se afla majoritatea operatiilor logice si a interfetei vizuale. În cadrul acestei aplicații sunt gestionate datele sportivilor, abonamentele, prezențele și celelalte funcționalități necesare administrării sălii de kickbox. De asemenea, aplicația oferă utilizatorului o interfață intuitivă prin care poate accesa și modifica informațiile stocate în sistem.

Odata cu logarea in aplicatie ,un server HTTP este pornit pe aplicatie ce va asculta mereu pentru cerei .Serverul are rolul de a primi date de la a doua componenta si a oferi aceste date aplicatiei WPF pentru procesare.

A doua componenta este API-ul care are un rol mai mult ca aplicatie client, el reprezentand intermediarul dintre utilizator/sportiv si aplicatia desktop.Pe el aflandu-se pagina web pentru prezenta sportivului si un controler ce trimite informatia oferita de utilizator catre serverul HTTP ce ruleaza pe aplicatia WPF.

Dupa ce aplicatia desktop primeste informatiile de la API aceasta verifica in baza de date, cu ajutor informatilor primite daca sportivul are abonamentul valid sau nu.In functie de rezultatul oferit de aplicatia WPF dupa procesearea informatiilor ,aplicatia WPF transmite un mesaj corespunzator API-ului care apoi este afisat pe pagina web a utilizatorului.In figura de mai jos este o schema ce arata comunicarea dintre cele doua aplicatii.



Figură 12 DIAGRAMA COMUNICARE APLICATII

Odată cu pornirea aplicației desktop, aceasta se asigură că există o bază de date SQLite. Acest 22isposi este realizat automat pentru a elimina necesitatea configurării 22ispos din partea utilizatorului și pentru a simplifica instalarea aplicației. Dacă baza de date nu există, aplicația WPF o creează automat și inițializează structura necesară funcționării sistemului.

După crearea bazei de date, aplicația adaugă contul de administrator al aplicației cu numele de utilizator „somo” și parola „kickbox”, pe care apoi îl salvează în baza de date nou 23ispos. Acest cont are rolul de a permite accesul 23isposi în aplicație și administrarea informațiilor stocate în 23ispos. Prin existența unui cont implicit de administrator se asigură că aplicația poate fi utilizată imediat după instalare, fără a fi necesară configurarea 23ispos a unui utilizator.

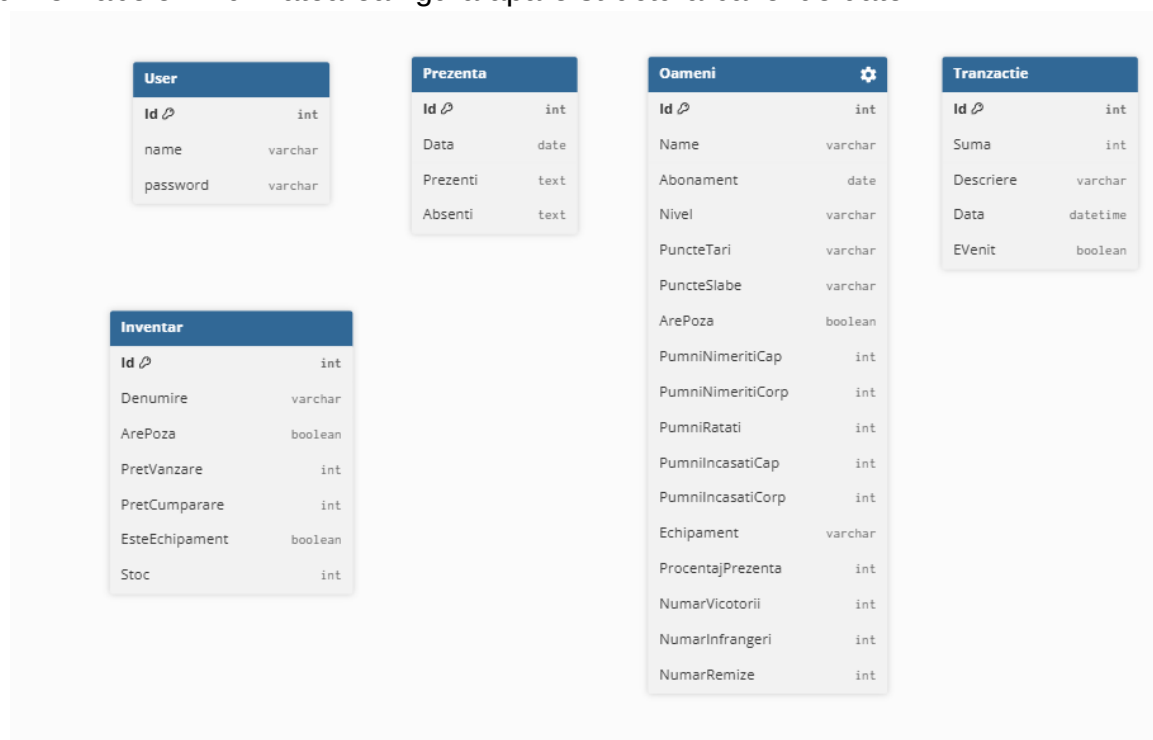
Un al doilea lucru pe care aplicația îl face la pornire este verificarea existenței unui mediu virtual Python (.venv). Dacă acesta nu există, aplicația îl creează automat pentru a pregăti mediul necesar executării scripturilor Python utilizate în cadrul proiectului. Utilizarea unui mediu virtual permite izolarea dependențelor necesare aplicației și evitarea conflictelor cu alte versiuni de pachete instalate pe sistemul de operare.

Acest mediu virtual este necesar pentru a putea rula scriptul Python direct din aplicația WPF. Scriptul Python este folosit pentru funcționalitățile de inteligență artificială implementate în cadrul proiectului, precum procesarea imaginilor și identificarea 23isposi a tehnicilor de kickbox. Prin integrarea limbajului Python cu aplicația desktop se pot utiliza biblioteci specializate în domeniul inteligenței artificiale și al procesării imaginilor.

Toate aceste lucruri sunt posibile cu ajutorul librăriei Csnakes. Aceasta oferă mecanismele necesare pentru comunicarea dintre aplicația dezvoltată în C# și scripturile Python, permițând apelarea funcțiilor Python direct din codul aplicației WPF.

3.3 STRUCTURA BAZEI DE DATE

Baza de date este de tip SQLite si in acest proiect este un fisier numit „DataFile.db” format din 5 Tabele .In urmatoarea figura apare structura bazei de date



Figură 13 DIAGRAMA STRUCTURA BAZA DE DATE

Tabelul Oameni conține informațiile despre sportivii înregistrați în cadrul clubului de kickbox. Acesta stochează date generale precum numele, data expirării abonamentului și nivelul sportivului. De asemenea, sunt păstrate statistici generate în urma analizelor video, precum numărul loviturilor nimerite sau încasate. El include și informații despre prezență, echipament și rezultatele obținute în competiții.

Tabelul Prezenta este utilizat pentru gestionarea participării sportivilor la antrenamente. Pentru fiecare zi sunt stocate listele cu persoanele prezente și absente. Informațiile din acest tabel sunt folosite pentru monitorizarea activității sportivilor și calcularea procentajului de prezență.

Tabelul Inventar conține informații despre echipamentele și produsele administrate de sală. Pentru fiecare articol sunt stocate denumirea, prețul de cumpărare, prețul de vânzare și stocul disponibil. Acest tabel permite gestionarea eficientă a resurselor materiale și urmărirea produselor existente.

Tabelul User este utilizat pentru stocarea conturilor care au acces la aplicație. Acesta conține numele de utilizator și parola necesare autentificării. Prin intermediul acestui tabel este controlat accesul la funcționalitățile sistemului.

Tabelul *Tranzactie* este folosit pentru evidența operațiunilor financiare realizate în cadrul clubului. Acesta stochează suma, descrierea și data fiecărei tranzacții. Cu ajutorul câmpului care indică dacă operațiunea reprezintă un venit sau o cheltuială, aplicația poate urmări situația financiară a sălii.

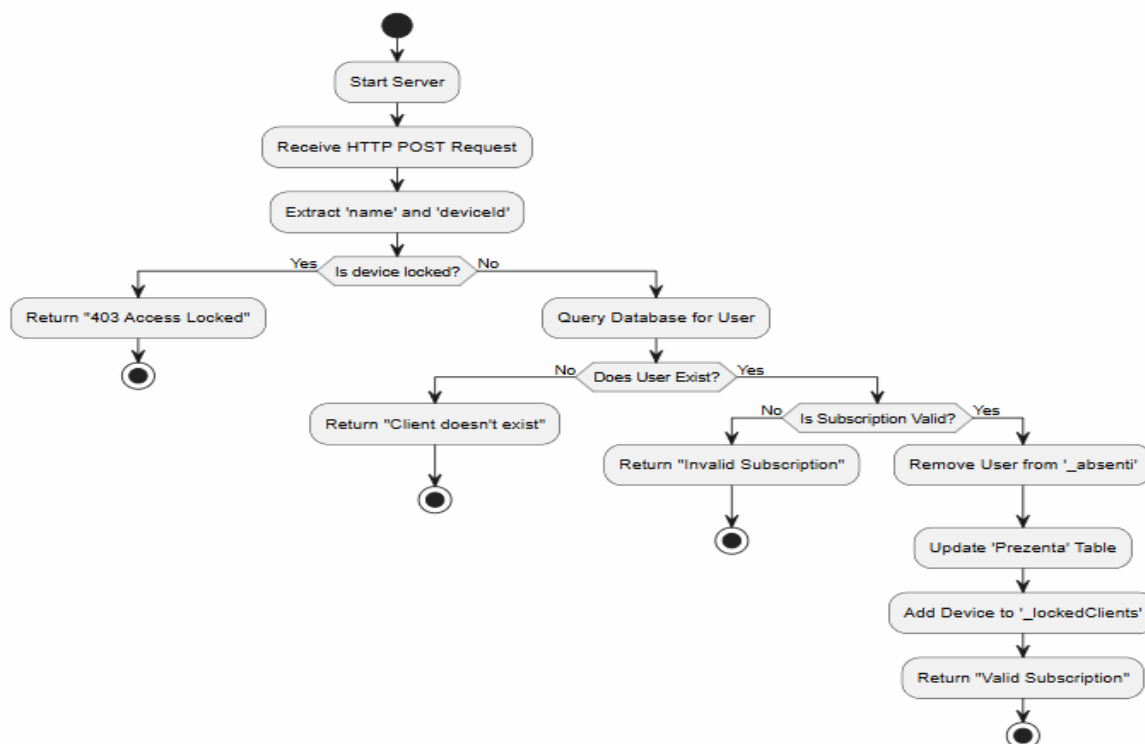
3.4 SCHEME SI DIAGrame FUNCTIONALE ALE APLICATIEI

În acest subcapitol voi prezenta modul în care anumite părți ale aplicației funcționează, utilizând diagrame ușor de înțeles. Aceste diagrame au rolul de a evidenția fluxul de date și interacțiunea dintre componentele sistemului, facilitând o mai bună înțelegere a arhitecturii aplicației și a modului în care utilizatorul interacționează cu aceasta.

3.4.1 DIAGRAMA DE ACTIVITATE A SERVERULUI HTTP

Clasa `HttpServer` este responsabilă pentru gestionarea unui server HTTP local, utilizat pentru a primi și procesa cereri HTTP. Aceasta este integrată în aplicația WPF și interacționează cu baza de date și cu `MainWindowViewModel` pentru a valida abonamentele utilizatorilor și pentru a actualiza prezența acestora.

În figura de mai jos se poate observa întreaga diagrama de activitate a serverului HTTP.



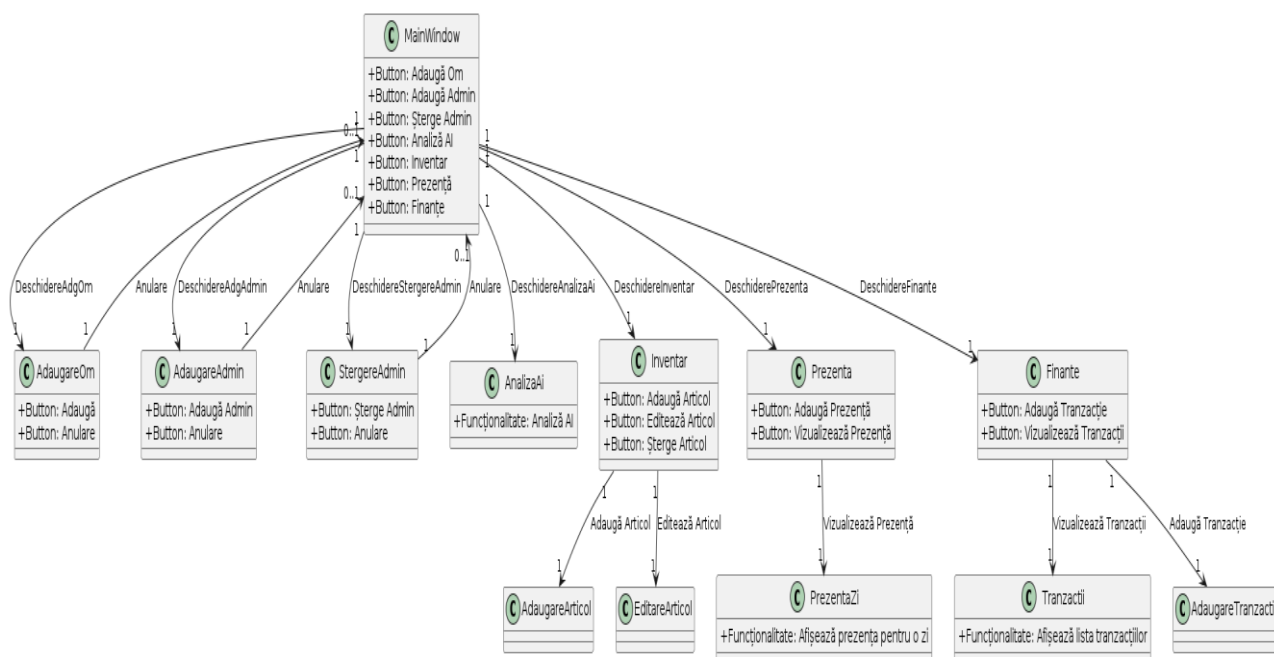
Figură 14 DIAGRAMA ACTIVITATE SERVER HTTP

Când serverul primește o cerere, verifică dacă aceasta este validă și dacă utilizatorul există în baza de date. Dacă utilizatorul este găsit, iar abonamentul său este 25ispos, serverul îl marchează ca 25isposi și actualizează lista de absenți. În același timp, dispozitivul utilizatorului este blocat temporar pentru a preveni validări multiple. Dacă utilizatorul nu este găsit sau abonamentul său este expirat, serverul trimite un mesaj corespunzător.

Pentru a realiza aceste operațiuni, clasa folosește alte componente ale aplicației. De exemplu, MainWindowViewModel este folosit pentru a accesa lista de absenți și pentru a actualiza interfața principală a aplicației. De asemenea, Bazadateconnect este clasa care gestionează conexiunea cu baza de date SQLite, unde sunt stocate informațiile despre utilizatori și prezență.

Pe scurt, HttpServer este responsabil pentru comunicarea dintre aplicație și alte 25ispositive, validarea utilizatorilor și actualizarea datelor în baza de date. Aceasta este o parte esențială a aplicației, care asigură funcționarea corectă a sistemului de prezență și abonamente.

3.4.2 SCHEMA FUNCTIONALA A APLICATIEI



Figură 15 SCHEMA FUNCTIONAL APLICATIE

Aplicația este structurată în mai multe ferestre, fiecare având un scop specific și funcționalități bine definite. Fiecare fereastră este accesibilă prin intermediul butoanelor din fereastra principală sau din alte ferestre, iar funcționalitățile lor sunt implementate pentru a răspunde nevoilor administrative ale antrenorilor.

Fereastra MainWindow este punctul central al aplicației, oferind utilizatorilor acces la toate modulele principale. Aceasta conține butoane care deschid ferestrele pentru gestionarea utilizatorilor, prezenței, tranzacțiilor financiare, inventarului, detaliile sportivului și analiza AI. Fiecare buton este asociat unei comenzi care inițiază deschiderea unei ferestre specifice. Această fereastră este esențială pentru navigarea în aplicație și asigură o interfață intuitivă pentru utilizatori ea fiind mereu în modul fullscreen.

Fereastra AdaugareOm permite adăugarea unui nou utilizator în baza de date. Utilizatorul poate introduce informațiile necesare despre noul membru, iar prin apăsarea butonului „Adaugă”, datele sunt salvate în baza de date iar UI-ul este actualizat corespunzător cu noul sportiv. Dacă utilizatorul apasă butonul „Anulare”, fereastra se închide fără a salva modificările. Această fereastră este importantă pentru gestionarea eficientă a membrilor și pentru menținerea unei baze de date actualizate.

Fereastra AdaugareAdmin este utilizată pentru adăugarea unui administrator nou. Similar cu fereastra AdaugareOm, utilizatorul poate introduce informațiile necesare despre administrator, iar prin apăsarea butonului „Adaugă Admin”, datele sunt salvate în baza de date iar acum există un nou cont cu care se poate loga în aplicație în afara celui creat când aplicația este pornită pentru prima oară. Butonul „Anulare” permite închiderea ferestrei fără a salva

modificările. Această funcționalitate este destinată utilizatorilor cu drepturi administrative și este esențială pentru gestionarea permisiunilor în aplicație.

Fereastra StergereAdmin permite eliminarea unui administrator existent. Utilizatorul poate selecta administratorul pe care dorește să îl elimine, iar prin apăsarea butonului format dintr-o iconiță specifică stingerii, acesta este eliminat din baza de date și UI-ul este actualizat corespunzător. Butonul „Anulare” permite închiderea ferestrei fără a efectua modificări. Această fereastră este utilă pentru menținerea unei liste actualizate de administratori.

Fereastra Prezenta este utilizată pentru gestionarea prezenței membrilor. În această fereastră sunt reprezentate zilele în care aplicația a fost deschisă. Utilizatorul poate vizualiza prezența pentru o anumită zi prin apăsarea pe ziua dorită, acest lucru ducând la deschiderea ferestrei PrezentaZi. Această funcționalitate este esențială pentru monitorizarea activității membrilor și pentru generarea de rapoarte privind prezența.

Fereastra PrezentaZi afișează prezența membrilor pentru o anumită zi. Utilizatorul poate vedea detalii despre membrii prezenți și absenți, iar aceste informații sunt utile pentru analiza activității organizației. Această fereastră este o extensie a funcționalității oferite de fereastra Prezenta.

Fereastra Inventar permite gestionarea articolelor din inventar. Utilizatorul poate adăuga articole noi, edita articole existente sau șterge articole care nu mai sunt necesare. Această fereastră este importantă pentru organizațiile care doresc să își administreze resursele fizice într-un mod eficient și organizat.

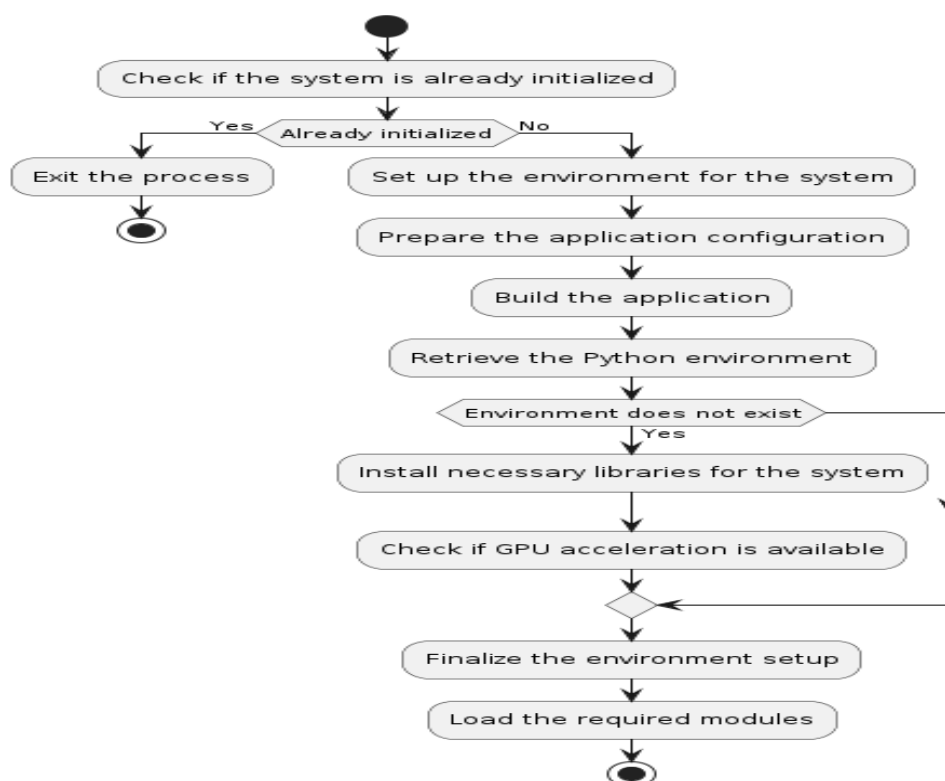
Fereastra Finante este utilizată pentru gestionarea tranzacțiilor financiare. Utilizatorul poate adăuga tranzacții suplimentare prin deschiderea ferestrei AdaugareTranzactieSuplimentara sau poate vizualiza istoricul tranzacțiilor prin deschiderea ferestrei Tranzactii. Această funcționalitate este esențială pentru monitorizarea și gestionarea resurselor financiare ale organizației.

Fereastra Tranzactii afișează lista tranzacțiilor financiare efectuate. Utilizatorul poate vedea detalii despre fiecare tranzacție, cum ar fi data, suma și descrierea acesteia. Această fereastră este utilă pentru analiza financiară și pentru generarea de rapoarte.

Fereastra AnalizaAi oferă funcționalități de analiză bazate pe inteligență artificială. Utilizatorul poate genera statistici și informații utile pentru luarea deciziilor. Această fereastră este o componentă avansată a aplicației, care adaugă valoare prin utilizarea tehnologiilor moderne.

Fiecare fereastră din aplicație este proiectată pentru a răspunde unor nevoi specifice, iar funcționalitățile lor sunt implementate cu respectarea reglementărilor privind protecția datelor și securitatea informațiilor. Aplicația este destinată utilizării interne și este accesibilă doar utilizatorilor autorizați.

3.4.3 DIAGRAMA DE ACTIVITATE A INTIALIZARI MEDIULUI PYTHON



Figură 16 DIAGRAMA ACTIVITATE INITIALIZARE PYTHON

În cadrul aplicației, există un proces esențial care pregătește un mediu necesar pentru integrarea scriptului de Python și bibliotecilor necesare folosirii acestuia. Totul începe cu verificarea dacă acest sistem a fost deja configurat. Dacă este cazul, procesul se oprește imediat. În schimb, dacă sistemul nu este inițializat, se trece la configurarea unui mediu izolat (.venv). Acest mediu este creat pentru a asigura că toate componentele funcționează într-un spațiu controlat, fără a afecta alte aplicații sau procese și se folosește de Csnakes pentru acest lucru.

După ce mediul este pregătit și aplicația este configurată pentru a utiliza acest spațiu, se aplică toate setările necesare pentru a permite funcționarea corectă a sistemului. În continuare, aplicația este construită și pregătită pentru a integra funcționalitățile externe. Acest pas este crucial pentru a asigura compatibilitatea între diferitele componente ale sistemului.

Dacă mediul izolat există, se trece la instalarea componentei PyTorch separat. PyTorch ne oferă folosirea unui set de instrumente care permit utilizarea unor funcționalități avansate, cum ar fi procesarea accelerată. Înainte de instalare, orice versiune anterioară a acestor instrumente este eliminată pentru a evita conflictele. Instalarea este realizată cu atenție, iar utilizatorul este informat despre succesul sau eșecul procesului. În cazul unui eșec, sunt oferite sugestii pentru remedierea problemei. Toate aceste lucruri se întâmplă deoarece dacă am instala bibliotecile după cum explică documentația Csnakes, ar instala versiune pentru CPU al PyTorch-ului ceea ce ar face analiza AI super încetă.

Un alt pas important este verificarea disponibilității unui accelerator grafic. Acest lucru este realizat prin rularea unor teste care determină dacă sistemul poate utiliza resursele hardware pentru a accelera procesarea. Dacă acceleratorul grafic este disponibil, utilizatorul este informat că aplicația va funcționa la performanță maximă și va începe instalarea versiunii PyTorch-ului ce permite procesarea accelerată folosind GPU. În caz contrar, utilizatorul este notificat că aplicația va utiliza procesorul principal, ceea ce poate duce la o viteză mai mică.

La final, mediul este configurat complet, iar aplicația este pregătită să utilizeze funcționalitățile externe. Acest proces asigură că toate componentele necesare sunt instalate și funcționează corect. Integrarea este realizată astfel încât să fie cât mai lină și eficientă. În ansamblu, acest proces este vital pentru a permite utilizarea unor funcționalități avansate într-un mod sigur și controlat.

4 IMPLEMENTARE

În acest capitol voi prezenta toți pașii pe care i-am parcurs în realizarea acestui proiect. De la crearea aplicației WPF, la crearea API-ului, antrenarea și implementarea elementului AI și implementarea bibliotecilor externe. Pentru fiecare etapă vor fi explicate bibliotecile și tehnologiile folosite, în același timp vor fi expuse exemple de cod sursă al acestor implementări.

4.1 IMPLEMENTAREA AI

În acest subcapitol se va prezenta totalitatea pașilor parcurși în antrenarea, testarea și implementarea elementului de analiză AI al unui meci/sparing de box.

Vor fi prezentate etapele colectării de date și pregătirea datelor pentru antrenarea modelului.

O să fie arată etapa de antrenare a modelului de YOLO personalizat pentru dorințele noastre.

Va fi prezentată și explicată crearea scriptului python principal al detecției pumnilor dintr-un video cu toate funcțiile folosite și uzul lor în analiza finală.

Se va explica cum a fost Python și AI-ul implementat în aplicația desktop WPF folosind biblioteca CSnakes.

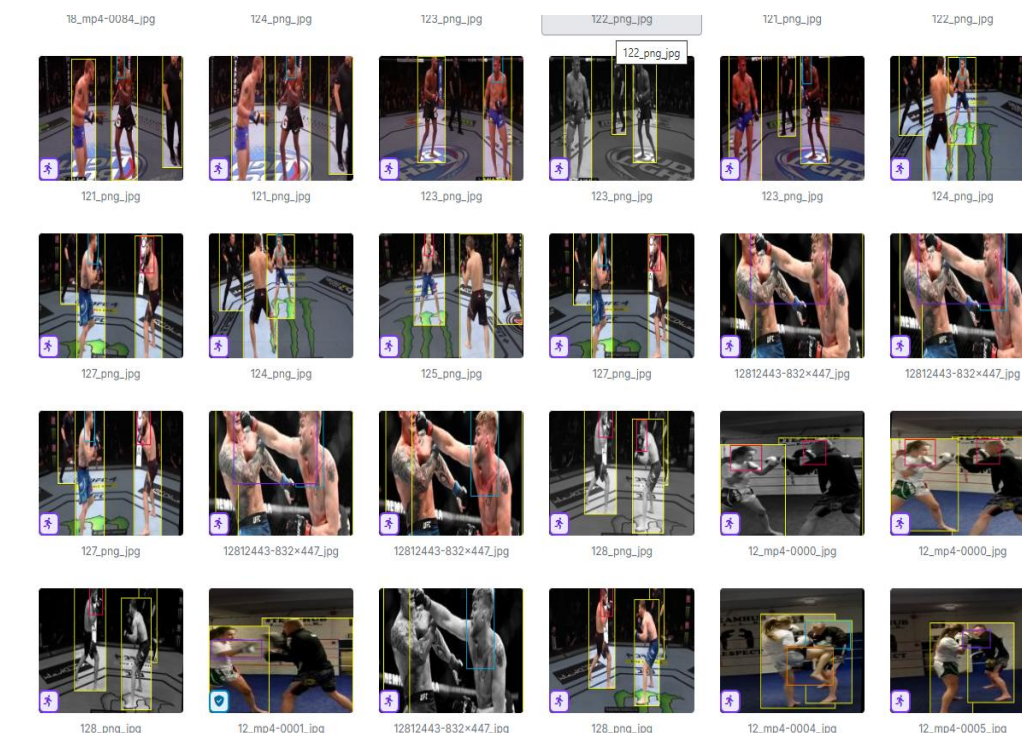
Pe final se vor prezenta rezultatele unei analize AI și explicarea elementelor din imagine.

4.1.1 COLECTAREA ȘI PREGĂTIREA DATELOR

Pentru colectarea datelor folosite în antrenarea AI-ului am folosit platforma Roboflow de unde am folosit un dataset cu diferite imagini în care apar doi luptători în mai multe condiții de lumină, mai multe unghiuri, mai multe tipuri de ringuri. Diversitatea imaginilor și condițiilor în care se afla luptătorii ajută AI-ul să înțeleagă ce se întâmplă în cât mai multe situații reale, ajutând astfel capacitatea de generalizare a acestuia.

Înainte de procesul de antrenare, imaginile au fost etichetate prin intermediul bounding box-urilor, fiecare lovitură fiind încadrată într-un dreptunghi care indică poziția sa exactă în imagine. Aceste etichete reprezintă informațiile pe baza cărora modelul învață să detecteze și să localizeze sportivii în cadrul unui cadru video. Utilizarea unui număr mare de imagini etichetate contribuie la obținerea unei precizii mai ridicate și la reducerea erorilor de detecție în timpul utilizării aplicației.

În imaginea de mai jos putem vedea un set de imagini din acest dataset cu bounding box-urile marcate desenate pe ele. Aceste marcaje evidențiază obiectele de interes pe care AI-ul trebuie să le identifice în cazul de față fiind pumni dați de luptător.



Figură 17 COLECTARE DATE

Dupa tot din platforma roboflow am putut sa trecem toate imaginile prin diferite efecte cum ar fi intoarcerea lor in diferite unghiuri, zoom in in locuri aleatorii in imagine, blurarea lor, si multe altele pentru a putea ajuta AI-ul sa inteleaga ce se intampla in mai multe conditii in care ar putea avea loc un meci/sparring.

Acest proces este cunoscut sub numele de augmentare a datelor și are rolul de a crește diversitatea imaginilor folosite în procesul de antrenare, fără a fi necesară colectarea manuală a unui număr foarte mare de imagini noi. Cu ajutorul aceste augmentari modelul este expus la mai multe variante ale aceleiași situații, ceea ce îl ajută să devină mai stabil și mai precis atunci când este folosit pe imagini sau secvențe video reale scazand rata de non detectii.

Acest lucru a marit numarul de imagini din dataset ceea ce a dus la un timp de antrenare mai mare. Totuși, creșterea volumului de date a fost necesară deoarece un dataset mai complex permite modelului să învețe mai multe caracteristici și să se adapteze mai bine la diferite condiții întâlnite în timpul unui meci sau sparring. Prin folosirea acestor metode de prelucrare a imaginilor, AI-ul poate să recunoască luptătorii chiar și atunci când apar schimbări de poziție, iluminare, perspectivă sau calitate a imaginii. De asemenea, augmentarea datelor contribuie la reducerea riscului ca modelul să învețe doar exemplele existente în dataset și să aibă performanțe slabe pe imagini noi.

De asemenea, pe lângă imaginile propriu-zise sunt create și fișierele de configurare necesare pentru modelul YOLO, acestea conținând informații despre locația datasetului și clasele care trebuie identificate acest fișier fiind numit data.yaml. Organizarea corectă a acestor fișiere este importantă deoarece modelul are nevoie de aceste informații pentru a putea procesa imaginile.

După aceea cu următoarea secțiune de cod vom da start antrenării propriu-zise a acestui AI. În această etapă modelul YOLO va începe să analizeze imaginile din dataset și să învețe să recunoască luptătorii folosind informațiile oferite prin bounding box-uri. Procesul de antrenare presupune mai multe treceri prin întregul set de date și este costisitor din punct de vedere al timpului și al stresului pus pe GPU/CPU.

În imaginea de mai jos este codul care da startul antrenării.

```
import os

# path to yolov5 folder
YOLO_PATH = "yolov5"

dataset_path = "dataset/data.yaml"

💡

command = f"""
python {YOLO_PATH}/train.py \
--img 640 \
--batch 16 \
--epochs 100 \
--data {dataset_path} \
--weights yolov5s.pt \
--name boxing_ai_model
"""

os.system(command)
```

Figură 20 COD ANTRENARE AI

În această secțiune de cod este reprezentat scriptul Python care porneste antrenarea AI-ului. În prima parte a codului într-o variabilă YOLO_PATH punem locația framework-ului YOLOv5. Această variabilă este folosită pentru ca programul să știe unde se află fișierele necesare pentru executarea procesului de antrenare. Prin această metodă putem păstra codul mai organizat și putem schimba mai ușor locația framework-ului fără să modificăm întreaga comandă de rulare.

Următoarea variabilă, dataset_path, reprezintă locația fișierului data.yaml generat împreună cu datasetul descărcat de pe platforma Roboflow. Acest fișier conține informații importante despre structura datasetului, precum locația imaginilor de antrenare, validare și testare, dar și clasele care trebuie recunoscute de către model. Acest fișier este foarte

important deoarece fara el modelul YOLO nu ar sti de unde sa incarce imaginile si ce obiecte trebuie sa invete sa detecteze in timpul procesului de antrenare.

Dupa aceea vine variabila principala, variabila command, ea fiind o comanda pe care o vom rula folosind libraria os. Aceasta variabila contine toate setarile necesare pentru pornirea procesului de antrenare si face legatura dintre proiectul nostru Python si framework-ul YOLOv5. Prin aceasta comanda putem controla modul in care modelul este antrenat si putem modifica diferiti parametri in functie de necesitatile proiectului.

Ea are mai multe proprietati una dintre ele este --img 640, care stabileste dimensiunea imaginilor folosite in antrenare. Alegerea acestei valori influenteaza atat precizia modelului cat si timpul necesar pentru procesarea imaginilor, deoarece imaginile cu o rezolutie mai mare necesita mai multe resurse de calcul. O dimensiune de 640 de pixeli reprezinta un compromis intre viteza de antrenare si calitatea rezultatelor obtinute.

O a doua proprietate este – batch 16 care defineste numarul de imagini procesate simultan intr-un singur pas de antrenare, validare si testare. Acest parametru are legatura directa cu memoria disponibila a calculatorului, deoarece un batch mai mare necesita mai multe resurse hardware, dar poate ajuta modelul sa invete mai stabil.

Proprietatea –epoch 100 stabileste numarul total de treceri prin intregul dataset, acest element ne arata timpul de antrenare cu cat acesta este mai mare antrenamentul va dura mai mult. Fiecare epoch reprezinta o trecere completa prin toate imaginile disponibile din dataset, iar dupa fiecare trecere modelul isi ajusteaza parametrii pentru a putea face detectii mai precise. Alegerea unui numar potrivit de epoch-uri este importanta deoarece un numar prea mic poate duce la un model care nu a invatat suficient, iar un numar prea mare poate duce la supraantrenare.

In final, dupa ce toate aceste proprietati sunt stabilite, comanda este executata si incepe procesul de antrenare al modelului YOLOv5. In urma acestui proces sunt generate fisierele rezultate care contin modelul antrenat, acesta putand fi folosit ulterior pentru detectarea automata a luptatorilor in imagini sau videoclipuri el este reprezentat de un fisier cu ce se termina in “.pt” si are numele pe care l-am introdus in comanda.

4.1.3 SCRIPTUL PYTHON

În această secțiune este prezentat modul în care a fost realizat programul responsabil pentru analiza automată a unui meci de box folosind modele de inteligență artificială. Acestea fiind modelul YOLO antrenat de noi si modelul SAM2 care este folosit pentru segmentarea luptatorilor. Aceste două modele lucrează împreună pentru a putea determina cine execută lovitura, cine o primește și zona în care aceasta este aplicată.

La începutul programului sunt importate bibliotecile necesare, precum YOLO pentru rularea modelului de detecție a loviturilor, SAM2VideoPredictor pentru segmentarea și urmărirea luptătorilor, OpenCV pentru procesarea videoclipurilor, NumPy pentru operații matematice asupra imaginilor, biblioteca JSON pentru extragerea rezultatelor la finalul analizei.

Prima componentă importantă a programului este încărcarea modelului YOLO prin linia `punch_model = YOLO('topaiever.pt')`. Acest model reprezintă modelul AI antrenat

special pentru detectarea loviturilor de box. În timpul procesării fiecărui cadru video, acesta analizează imaginea și identifică zonele unde apare o posibilă lovitură, returnând poziția acesteia sub forma unui bounding box.

Funcția `box_iou_with_mask()` are rolul de a calcula cât de mult se suprapune zona unei lovituri detectate peste masca unui luptător. Această informație este importantă deoarece ajută programul să stabilească dacă lovitura aparține unui anumit sportiv. Practic, funcția compară poziția loviturii identificare de YOLO cu poziția corpului identificată de modelul SAM2. În următoarea imagine avem codul acestei funcții.

```
def box_iou_with_mask(box, mask):  
    x1, y1, x2, y2 = [int(v) for v in box]  
    x1 = max(0, x1)  
    y1 = max(0, y1)  
    x2 = min(mask.shape[1], x2)  
    y2 = min(mask.shape[0], y2)  
  
    if x2 <= x1 or y2 <= y1:  
        return 0.0  
  
    crop = mask[y1:y2, x1:x2]  
    overlap_pixels = int(crop.sum())  
    box_pixels = (x2 - x1) * (y2 - y1)  
  
    return overlap_pixels / max(1, box_pixels)
```

Figură 21 OVERLAP MASCA AI

Funcția `get_head_box()` este utilizată pentru determinarea zonei capului unui luptător. Aceasta analizează masca generată de SAM2 și calculează o zonă aproximativă corespunzătoare capului, folosind partea superioară a corpului detectat. Această zonă este folosită pentru a putea diferenția loviturile aplicate la cap față de cele aplicate în corp. În următoarea imagine avem codul acestei funcții.

```
def get_head_box(mask, head_fraction=0.25):  
    ys, xs = np.where(mask > 0)  
    if len(ys) == 0:  
        return None  
  
    y_min = int(ys.min())  
    y_max = int(ys.max())  
    x_min = int(xs.min())  
    x_max = int(xs.max())  
  
    head_y_max = y_min + int((y_max - y_min) * head_fraction)  
    return (x_min, y_min, x_max, head_y_max)
```

Figură 22 AFLAREA CAPULUI AI

Funcția `get_body_box()` funcționează într-un mod asemănător, dar determină zona corpului luptătorului. Prin împărțirea măștii în zone diferite, programul poate clasifica loviturile și poate oferi statistici separate pentru lovituri la cap și lovituri la corp.

Funcția `box_overlap_fraction()` este folosită pentru calcularea gradului de suprapunere dintre două zone. În cadrul analizei este utilizată pentru a determina dacă o lovitură detectată intersectează suficient zona capului sau a corpului pentru a fi considerată o lovitură reușită. În următoarea imagine avem codul acestei funcții.

```
def box_overlap_fraction(box_a, box_b):  
    """Fraction of box_a that overlaps with box_b."""  
    ax1, ay1, ax2, ay2 = [int(v) for v in box_a]  
    bx1, by1, bx2, by2 = [int(v) for v in box_b]  
  
    ix1 = max(ax1, bx1)  
    iy1 = max(ay1, by1)  
    ix2 = min(ax2, bx2)  
    iy2 = min(ay2, by2)  
  
    if ix2 <= ix1 or iy2 <= iy1:  
        return 0.0  
  
    inter = (ix2 - ix1) * (iy2 - iy1)  
    area_a = max(1, (ax2 - ax1) * (ay2 - ay1))  
    return inter / area_a
```

Figură 23 OVERLAP CUTII AI

Funcția `draw_mask_overlay()` are rol vizual și este folosită pentru afișarea măștilor generate de SAM2 peste imaginea originală. Aceasta permite utilizatorului să vadă zonele detectate de inteligența artificială și să observe modul în care modelul urmărește fiecare luptător.

Funcția `get_box_center()` calculează centrul unui bounding box. Această informație este utilizată ulterior pentru urmărirea direcției de deplasare a unei lovituri și pentru determinarea luptătorului care a inițiat atacul. În următoarea imagine avem codul acestei funcții.

```
def get_box_center(box):  
    x1, y1, x2, y2 = box  
    return ((x1 + x2) / 2, (y1 + y2) / 2)
```

Figură 24 CENTRUL CUTII AI

Funcția principală a întregului sistem este `analyze_fight()`. Aceasta primește videoclipul de analizat, numele celor doi luptători și locația unde va fi salvat videoclipul rezultat. În interiorul acestei funcții sunt combinate toate componentele AI pentru realizarea analizei complete.

În cadrul funcției `attribute_punch()` este realizată logica principală pentru atribuirea unei lovituri. Programul verifică poziția loviturii detectate, suprapunerea acesteia cu măștile

luptătorilor și direcția de deplasare pentru a determina cine este atacatorul și cine este apărătorul. După identificarea celor doi participanți, funcția stabilește și tipul loviturii, respectiv dacă aceasta este o lovitură la cap sau la corp.

Funcția `register_hit()` are rolul de a salva rezultatele unei lovituri confirmate. Aceasta actualizează statisticile fiecărui luptător, crescând numărul de lovituri aplicate sau primite. Pentru a evita numărarea aceleiași lovituri de mai multe ori, este utilizat un sistem de cooldown între detecții.

Funcția `register_missed_punch()` este folosită pentru evidențierea loviturilor ratate. Dacă modelul detectează o lovitură dar nu găsește un contact cu celălalt luptător, aceasta este salvată ca o lovitură nereușită.

După selectarea manuală a celor doi luptători din primul cadru video, modelul SAM2 începe procesul de segmentare și urmărire a acestora pe toate cadrele videoclipului. În următoarea imagine apare codul pentru selectarea fiecărui luptător din primul frame al videoclipului.

```
cap = cv2.VideoCapture(video_path)

ret, frame0 = cap.read()

if not ret:
    raise Exception("Could not read video")

cap.release()

x, y, w, h = cv2.selectROI(
    f"Select {fighter1_name}",
    frame0,
    showCrosshair=False
)

x2, y2, w2, h2 = cv2.selectROI(
    f"Select {fighter2_name}",
    frame0,
    showCrosshair=False
)

cv2.destroyAllWindows()

bboxes = [
    [x, y, x + w, y + h],
    [x2, y2, x2 + w2, y2 + h2]
]
```

Figură 25 SELECTARE LUPTATORI AI

Pentru fiecare cadru, programul extrage măștile luptătorilor, rulează modelul YOLO pentru detectarea loviturilor și combină rezultatele celor două modele pentru a realiza analiza.

La final, programul desenează peste videoclip informațiile obținute: poziția luptătorilor, loviturile detectate, tipul acestora și statistici precum numărul de lovituri aplicate sau primite și la final trimite statisticile în format JSON aplicației desktop.

4.1.4 IMPLEMENTARE IN WPF

Pentru implementarea in WPF am folosit biblioteca CSnakes. Prima oara in Visual Studio, cu ajutor managerului oficial de pachete NutGet.Acesta este un instrument integrat care simplifica si automatizeaza descarcarea dependentelor in proiectele .NET. Am instalat aceasta dependenta in proiect doar prin cateva clicuri.

Prima oara am creat o clasa de initializare numita CSnakesInitialization care are scopul principal sa configureze mediul Python in interiorul aplicatiei WPF folosind biblioteca CSnakes.In imaginea de mai jos apare o prima secventa de cod al aceste clase.

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.IO;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Text;
using System.Threading.Tasks;
using CSnakes.Runtime;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;

namespace Modern
{
    4 references
    public class CSnakesInitialization
    {
        private static IPythonEnvironment? _env;
        public static IDemo? _module;

        1 reference
        public static void InitializeSnakes()
        {
            if (_module != null)
                return;

            string venvPath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, ".venv");

            var builder = Host.CreateApplicationBuilder();
            var home = Path.Join(Environment.CurrentDirectory, ".");

            builder.Services
                .WithPython()
                .WithHome(home)
                .WithVirtualEnvironment(venvPath)
                .WithPipInstaller()
                .FromRedistributable();

            var app = builder.Build();
            var env = app.Services.GetRequiredService<IPythonEnvironment>();

            _env = env;

            _module = env.Demo();
        }
    }
}
```

Figură 26 CLASA CSNAKESINITIALIZATION

Această secvență de cod se ocupă de inițializarea unui mediu Python integrat în aplicație. Mai întâi, verifică dacă modulul este deja inițializat (`_module != null`), iar dacă este, procesul se oprește pentru a evita reinițializarea. În continuare, definește calea către un mediu virtual Python (`.venv`) în directorul aplicației. Se creează un constructor pentru aplicație, iar serviciile sunt configurate pentru a include suport Python, specificând locația directorului de lucru, utilizarea unui mediu virtual și instalarea unui manager de pachete (`pip`). După configurare, aplicația este construită, iar serviciul de mediu Python este obținut din serviciile aplicației. Acest mediu este apoi stocat într-o variabilă statică (`_env`), iar modulul Python necesar este încărcat și stocat în variabila `_module`, pregătind astfel aplicația pentru utilizarea funcționalităților Python.

```
static bool RunPipCommand(string pipPath, string arguments, bool showOutput = false)
{
    var process = new Process
    {
        StartInfo = new ProcessStartInfo
        {
            FileName = pipPath,
            Arguments = arguments,
            RedirectStandardOutput = true,
            RedirectStandardError = true,
            UseShellExecute = false,
            CreateNoWindow = true
        }
    };

    if (showOutput)
    {
        process.OutputDataReceived += (sender, e) =>
        {
            if (!string.IsNullOrEmpty(e.Data))
                Console.WriteLine($" {e.Data}");
        };

        process.ErrorDataReceived += (sender, e) =>
        {
            if (!string.IsNullOrEmpty(e.Data))
                Console.WriteLine($" {e.Data}");
        };
    }

    process.Start();

    if (showOutput)
    {
        process.BeginOutputReadLine();
        process.BeginErrorReadLine();
    }

    process.WaitForExit();

    return process.ExitCode == 0;
}
```

Figură 27 CLASA RUNPIPCOMMAND

Această funcție este un utilitar general pentru rularea comenzilor `pip` în mediul virtual configurat. Primește ca parametri calea către executabilul `pip`, argumentele comenzii și o opțiune pentru a afișa sau nu ieșirea comenzii în consolă. Funcția creează un proces nou pentru a executa comanda `pip`, configurând redirecționarea ieșirii standard și a erorilor standard pentru a captura mesajele generate de comandă. Dacă opțiunea de afișare este

activată, ieșirea și erorile sunt afișate în timp real în consolă. După ce procesul se încheie, funcția returnează un rezultat boolean care indică succesul sau eșecul comenzii. Această funcție este utilizată pentru a instala, dezinstala sau actualiza pachete Python în mediul virtual, fiind un element esențial pentru gestionarea dependențelor aplicației.

```
public static void InstallPyTorchWithCuda()
{
    var venvPath = Path.Join(Environment.CurrentDirectory, ".", ".venv");
    var pipPath = OperatingSystem.IsWindows()
        ? Path.Join(venvPath, "Scripts", "pip.exe")
        : Path.Join(venvPath, "bin", "pip3");

    if (!File.Exists(pipPath))
    {
        return;
    }

    RunPipCommand(pipPath, "uninstall torch torchvision torchaudio -y", showOutput: false);

    bool success = RunPipCommand(
        pipPath,
        "install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu126 --force-reinstall --no-cache-dir",
        showOutput: true
    );
}
```

Figură 28 INSTALARE PYTHORCH

Această funcție este responsabilă pentru instalarea bibliotecii PyTorch cu suport pentru CUDA, care permite utilizarea accelerării GPU. Mai întâi, verifică dacă managerul de pachete pip este disponibil în mediul virtual configurat. Dacă pip nu este găsit, funcția se oprește, deoarece instalarea nu poate continua. Înainte de instalarea propriu-zisă, orice versiune existentă de PyTorch este dezinstaltată pentru a asigura un mediu curat și a evita conflictele. Apoi, funcția rulează o comandă pip pentru a instala PyTorch, împreună cu alte componente necesare, cum ar fi torchvision și torchaudio, folosind o sursă specifică pentru pachetele cu suport CUDA. Această funcție este esențială pentru pregătirea mediului Python astfel încât să poată utiliza GPU-ul pentru sarcini intensive de calcul.

```
public static bool VerifyCudaAvailability()
{
    var venvPath = Path.Join(Environment.CurrentDirectory, ".", ".venv");
    var pythonPath = OperatingSystem.IsWindows()
        ? Path.Join(venvPath, "Scripts", "python.exe")
        : Path.Join(venvPath, "bin", "python3");

    if (!File.Exists(pythonPath))
    {
        return false;
    }

    var verifyProcess = new Process
    {
        StartInfo = new ProcessStartInfo
        {
            FileName = pythonPath,
            Arguments = "-c \"import torch; print(f'PyTorch version: {torch.__version__}'); print(f'CUDA availa",
            RedirectStandardOutput = true,
            RedirectStandardError = true,
            UseShellExecute = false,
            CreateNoWindow = true
        }
    };

    verifyProcess.Start();
    string output = verifyProcess.StandardOutput.ReadToEnd();
    string error = verifyProcess.StandardError.ReadToEnd();
    verifyProcess.WaitForExit();

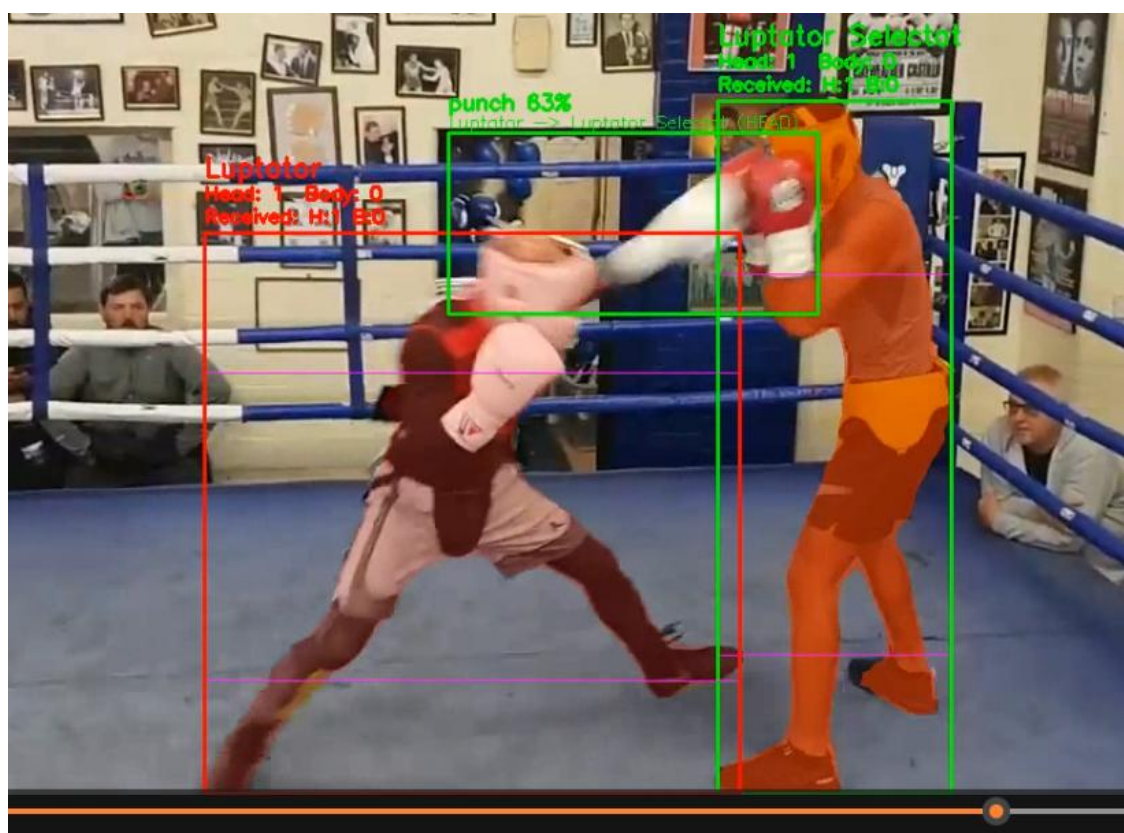
    Console.WriteLine(output);

    if (output.Contains("CUDA available: True"))
    {
        return true;
    }
    else
    {
        return false;
    }
}
```

Figură 29 VERIFICARE CUDA

Această funcție verifică dacă suportul pentru CUDA este disponibil în mediul configurat. Mai întâi, determină locația executabilului Python din mediul virtual. Dacă executabilul Python nu este găsit, funcția se oprește și returnează false, indicând că verificarea nu poate fi efectuată. În caz contrar, funcția rulează un script Python care utilizează biblioteca PyTorch pentru a verifica versiunea instalată, disponibilitatea suportului pentru CUDA, versiunea CUDA, numărul de GPU-uri disponibile și numele GPU-ului principal. Rezultatele sunt afișate în consolă, iar funcția returnează true dacă suportul pentru CUDA este activ. Această funcție este crucială pentru a determina dacă aplicația poate beneficia de accelerarea GPU sau dacă va trebui să utilizeze procesorul principal (CPU), ceea ce poate afecta performanța.

4.1.5 REZULTATE DUPA ANALIZA



Figură 30 REZULTATE AI

În imaginea de mai sus este prezentat un cadru extras din videoclipul analizat cu ajutorul inteligenței artificiale, în care pot fi observate segmentările celor doi luptători generate de modelul SAM2, utilizate pentru identificarea și urmărirea acestora. Totodată, este evidențiată detectarea unui pumn realizată de modelul personalizat YOLO, care a atribuit lovitura sportivului „Luptător”, în timp ce sportivul lovit este identificat ca „Luptător Selectat”. Acesta este evidențiat prin schimbarea culorii măștii sale în roșu, indicând faptul că sistemul a detectat și validat o lovitură asupra sa.

4.2 IMPLEMENTAREA APLICATIEI WPF

În acest capitol vor fi prezentate toate etapele parcurse în procesul de dezvoltare a aplicației desktop realizate în tehnologia WPF (Windows Presentation Foundation). Vor fi descrise atât aspectele legate de proiectarea și implementarea interfeței grafice, cât și componentele responsabile de logica aplicației, gestionarea datelor și comunicarea dintre module. De asemenea, vor fi explicate deciziile tehnice luate pe parcursul dezvoltării, tehnologiile și bibliotecile utilizate, precum și provocările întâlnite și soluțiile adoptate pentru rezolvarea acestora. Scopul acestui capitol este de a oferi o imagine de ansamblu asupra procesului de realizare a aplicației și de a evidenția principalele etape care au contribuit la obținerea rezultatului final.

4.2.1 IMPLEMENTARE BAZA DE DATE

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;

namespace Modern.Model
{
    65 references
    internal class Bazadateconnect:DbContext
    {
        0 references
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlite("Data Source = DataFile.db");
        }

        6 references
        public DbSet<User> Users { get; set; }

        16 references
        public DbSet<Oameni> Oameni { get; set; }

        7 references
        public DbSet<Inventar> Articole { get; set; }

        8 references
        public DbSet<Prezenta> Prezenti { get; set; }

        7 references
        public DbSet<Tranzactie> Tranzactii { get; set; }
    }
}
```

Figură 31 CONTEXT BAZA DE DATE

În imaginea de mai sus se afla clasa Bazadateconnect este contextul bazei de date și definește conexiunea la baza de date, precum și tabelele acesteia. Aceasta moștenește din DbContext și utilizează metoda OnConfiguring pentru a specifica locația bazei de date și alte setări necesare. Tabelele bazei de date sunt definite prin proprietăți de tip DbSet, fiecare corespunzând unei clase din folderul Model. Aceste clase definesc structura și relațiile dintre datele stocate.

Clasa User reprezintă tabela Users și este utilizată pentru gestionarea utilizatorilor aplicației. Fiecare utilizator are un identificator unic (Id), un nume (name) și o parolă (password). Această clasă este esențială pentru autentificarea utilizatorilor și pentru gestionarea accesului la aplicație. În mod similar, clasa Oameni definește tabela Oameni, care stochează informații despre membrii organizației. Aceasta include câmpuri precum numele membrului, data abonamentului, statistici despre performanță și alte detalii relevante.

Clasa Inventar definește tabela Articole, care stochează informații despre articolele din inventar. Fiecare articol are un nume, prețuri de vânzare și cumpărare, o cantitate disponibilă și alte detalii. Această clasă este utilizată pentru gestionarea resurselor fizice ale organizației. Clasa Prezenta definește tabela Prezenti, care stochează informații despre

prezența membrilor, inclusiv lista celor prezenți și absenți pentru o anumită zi. Aceasta este utilizată pentru monitorizarea activității membrilor.

Clasa Tranzactie definește tabela Tranzactii, care stochează informații despre tranzacțiile financiare. Fiecare tranzacție include suma, descrierea, data și un indicator care specifică dacă este un venit sau o cheltuială. Această clasă este esențială pentru gestionarea finanțelor organizației și pentru urmărirea fluxurilor financiare.

```
3 references
public partial class App : Application
{
    0 references
    protected override void OnStartup(StartupEventArgs e)
    {
        DatabaseFacade facade = new DatabaseFacade(new Bazadateconnect());
        facade.EnsureCreated();

        User somo = new User
        {
            name = "somo",
            password = "kickbox"
        };

        using (Bazadateconnect bz = new Bazadateconnect())
        {
            if (!bz.Users.Any())
            {
                bz.Users.Add(somo);
                bz.SaveChanges();
            }
        }
    }
}
```

Figură 32 CREARE BAZA DE DATE

În cadrul aplicației, fișierul App.xaml.cs reprezintă punctul de intrare al aplicației WPF și conține logica necesară pentru inițializarea acesteia. Metoda OnStartup este suprascrisă pentru a executa pașii necesari la pornirea aplicației. Primul pas este verificarea și crearea bazei de date. Se utilizează clasa Bazadateconnect, care reprezintă contextul bazei de date, iar prin intermediul metodei EnsureCreated se asigură că baza de date există. Dacă aceasta nu există, este creată automat pe baza configurației definite în clasele din folderul Model. Acest proces garantează că structura bazei de date este pregătită pentru utilizare.

După ce baza de date este creată, aplicația verifică dacă există utilizatori în tabela Users. Dacă tabela este goală, se adaugă un utilizator implicit cu numele "somo" și parola "kickbox". Acest utilizator este creat pentru a permite accesul inițial în aplicație și pentru a asigura funcționalitatea de bază. Utilizatorul este adăugat în baza de date folosind o conexiune deschisă prin clasa Bazadateconnect, iar modificările sunt salvate prin metoda SaveChanges. Acest pas este esențial pentru a popula baza de date cu date inițiale necesare.

4.2.2 IMPLEMENTARE LOGARE SI SERVER HTTP

```
namespace Modern.ViewModel
{
    2 references
    public class LoginCommand : CommandBase
    {
        private readonly LoginViewModel _viewModel;

        1 reference
        public LoginCommand(LoginViewModel viewModel)
        {
            _viewModel = viewModel;
        }

        1 reference
        public override void Execute(object parameter)
        {
            if (parameter is Window loginWindow)
            {
                var username = _viewModel.Username;
                var parola = _viewModel.Password;

                if (username == null || parola == null)
                {
                    return;
                }

                string good = username.ToString();
                good = good.Trim();

                using (Bazadateconnect context = new Bazadateconnect())
                {
                    bool userfound = context.Users.Any(user => user.name == good && user.password == parola);

                    if (userfound)
                    {
                        MainWindow mainWindow = new MainWindow();
                        mainWindow.Show();
                        loginWindow.Close();
                    }
                    else
                    {
                        _viewModel.Msgeroare = "Usernameul sau Parola este greșită";
                    }
                }
            }
        }
    }
}
```

Figură 33 COD LOGIN

Partea de logare în aplicație este gestionată prin clasa LoginCommand, care implementează logica necesară pentru autentificarea utilizatorilor. Aceasta este conectată la un LoginViewModel, care gestionează datele introduse de utilizator, cum ar fi numele de utilizator și parola, precum și mesajele de eroare afișate în interfața grafică. Procesul începe atunci când utilizatorul apasă butonul de logare, declanșând metoda Execute din LoginCommand.

În cadrul metodei Execute, datele introduse de utilizator sunt preluate din LoginViewModel. Numele de utilizator și parola sunt validate pentru a verifica dacă nu sunt nule. Dacă oricare dintre aceste valori este nulă, procesul de autentificare se oprește imediat, deoarece nu se poate continua fără date valide. După această verificare, numele de utilizator este curățat de spațiile albe suplimentare folosind metoda Trim, pentru a evita erorile cauzate de introducerea accidentală a spațiilor.

După preluarea și curățarea datelor, se deschide o conexiune la baza de date utilizând clasa Bazadateconnect. În această etapă, aplicația verifică dacă există un utilizator în tabela Users care are atât numele de utilizator, cât și parola corespunzătoare. Această

verificare este realizată prin metoda Any, care returnează true dacă utilizatorul este găsit și false în caz contrar. Această metodă asigură că autentificarea este rapidă și eficientă.

Dacă utilizatorul este găsit în baza de date, aplicația creează o nouă instanță a ferestrei principale (MainWindow) și o afișează utilizatorului. În același timp, fereastra de logare este închisă, iar utilizatorul este redirecționat către interfața principală a aplicației. În cazul în care utilizatorul nu este găsit, un mesaj de eroare este afișat utilizatorului. Acest mesaj este setat în proprietatea Msgeroare din LoginViewModel, iar interfața grafică este actualizată pentru a reflecta eroarea. În imaginea de mai jos apare și viewmodel-ul al pagini de logare.

```
namespace Modern.ViewModel
{
    5 references
    public class LoginViewModel : ViewModelBase
    {
        private string _username;

        2 references
        public string Username
        {
            get { return _username; }

            set
            {
                _username = value;
                OnPropertyChanged(nameof(Username));
            }
        }

        private string _password;

        3 references
        public string Password
        {
            get { return _password; }

            set
            {
                _password = value;
                OnPropertyChanged(nameof(Password));
            }
        }

        private string _msgeroare;

        2 references
        public string Msgeroare
        {
            get { return _msgeroare; }

            set
            {
                _msgeroare = value;
                OnPropertyChanged(nameof(Msgeroare));
            }
        }

        1 reference
        public ICommand LoginComand { get; }

        1 reference
        public LoginViewModel()
        {
            LoginComand = new LoginCommand(this);
        }
    }
}
```

Figură 34 VIEWMODEL LOGIN

Clasa HttpServer este responsabilă pentru implementarea unui server HTTP care permite aplicației să gestioneze cereri HTTP și să răspundă acestora. Aceasta utilizează clasa HttpListener pentru a asculta cererile pe o adresă specificată și este integrată cu logica aplicației prin intermediul clasei MainWindowViewModel. Serverul este proiectat să gestioneze cereri de tip POST și OPTIONS, oferind răspunsuri corespunzătoare în funcție de datele primite.

Constructorul clasei primește două argumente: un obiect HttpListener, care este utilizat pentru a asculta cererile HTTP, și o instanță a clasei MainWindowViewModel, care

permite serverului să acceseze datele și logica aplicației principale. Acestea sunt stocate în câmpurile private `_listener` și `_mainwindowviewmodel`, iar serverul este pregătit pentru a începe să asculte cererile.

Metoda `Start` inițializează serverul HTTP prin adăugarea unui prefix pentru URL-ul pe care serverul îl va asculta (de exemplu, `http://localhost:8000/server/`). Aceasta pornește ascultarea cererilor prin metoda `Start` a obiectului `HttpListener` și setează flag-ul `_isRunning` la `true`. De asemenea, pornește un proces asincron care rulează metoda `HandleRequest` pentru a gestiona cererile primite.

Metoda `HandleRequest` este responsabilă pentru gestionarea cererilor HTTP. Aceasta rulează într-un ciclu continuu atâta timp cât serverul este activ. Când o cerere este primită, metoda verifică tipul acesteia. Pentru cererile de tip `OPTIONS`, serverul răspunde imediat cu un cod de stare 200 (OK) și închide conexiunea. Pentru cererile de tip `POST`, serverul citește corpul cererii, deserializă datele JSON și prelucrează informațiile primite.

În cazul cererilor `POST`, serverul extrage informații precum numele utilizatorului (`name`) și identificatorul dispozitivului (`deviceId`). Acestea sunt utilizate pentru a crea un identificator unic pentru client (`clientKey`). Dacă clientul este deja blocat (de exemplu, a validat deja un utilizator), serverul răspunde cu un mesaj de eroare și un cod de stare 403 (Forbidden). În caz contrar, serverul verifică dacă utilizatorul există în baza de date și dacă abonamentul acestuia este valid. Dacă abonamentul este valid, utilizatorul este marcat ca prezent, iar informațiile sunt actualizate în baza de date.

Metoda `SendResponse` este utilizată pentru a trimite răspunsuri către client. Aceasta primește contextul cererii, un mesaj și un cod de stare HTTP. Mesajul este convertit în format text și trimis în fluxul de ieșire al răspunsului. Această metodă asigură că răspunsurile sunt trimise într-un format standardizat.

Clasa include, de asemenea, metode pentru deblocarea clienților (`UnlockClient`) și pentru deblocarea tuturor clienților (`UnlockAll`). Aceste metode sunt utile pentru gestionarea accesului clienților la server. În concluzie, clasa `HttpServer` oferă o implementare robustă și flexibilă a unui server HTTP, permițând aplicației să gestioneze cereri și să interacționeze cu alte sisteme prin intermediul unui protocol web. În imaginile de mai jos este prezentat codul.

```
namespace Modern.Model
{
    3 references
    public class HttpServer
    {
        private static HttpListener _listener;
        private MainWindowViewModel _mainwindowviewModel;
        private bool _isRunning;

        private static HashSet<string> _lockedClients = new HashSet<string>();

        1 reference
        public HttpServer(HttpListener listener, MainWindowViewModel mainwindowviewModel)
        {
            _listener = listener;
            _mainwindowviewModel = mainwindowviewModel;
        }

        1 reference
        private async Task HandleRequest()
        {
            while (_isRunning)
            {
                HttpListenerContext context = null;
                try
                {
                    context = await _listener.GetContextAsync();

                    context.Response.AddHeader("Access-Control-Allow-Origin", "*");
                    context.Response.AddHeader("Access-Control-Allow-Methods", "POST, GET, OPTIONS");
                    context.Response.AddHeader("Access-Control-Allow-Headers", "Content-Type");

                    if (context.Request.HttpMethod == "OPTIONS")
                    {
                        context.Response.StatusCode = 200;
                        context.Response.Close();
                        continue;
                    }

                    if (context.Request.HttpMethod == "POST")
                    {
                        Console.WriteLine("POST request received");

                        using (StreamReader reader = new StreamReader(context.Request.InputStream))
                        {
                            string body = await reader.ReadToEndAsync();

                            dynamic data = JsonConvert.DeserializeObject<object>(body);
                            string receivedName = data?.name;
                            string deviceId = data?.deviceId;

                            string clientKey = !string.IsNullOrEmpty(deviceId)
                                ? deviceId
                                : context.Request.RemoteEndPoint.Address.ToString();
                        }
                    }
                }
                catch { }
            }
        }
    }
}
```

Figură 35 COD SERVER HTTP pt 1

```
if (_lockedClients.Contains(clientKey))
{
    await SendResponse(context, "Access locked. You have already validated someone.", 403);
    context.Response.Close();
    continue;
}

using (Bazadateconnect bz = new Bazadateconnect())
{
    var people = bz.Oameni.ToList();
    var prezenta = bz.Prezenti.ToList();

    var person = people.FirstOrDefault(om =>
        om.Name.Equals(receivedName, StringComparison.InvariantCultureIgnoreCase));
    var prez = prezenta.FirstOrDefault(p => p.Data == DateOnly.FromDateTime(DateTime.Today));

    if (person != null)
    {
        if (DateOnly.FromDateTime(DateTime.Today) > person.Abonament.AddMonths(1))
        {
            await SendResponse(context, $"Hello {receivedName}! Invalid Subscription", 200);
        }
        else
        {
            // Valid - lock this device
            _lockedClients.Add(clientKey);
            await SendResponse(context, $"Hello {receivedName}! Valid Subscription", 200);

            foreach (var absent in _mainwindowviewModel._absenti)
            {
                if (absent == receivedName)
                {
                    _mainwindowviewModel._absenti.Remove(absent);
                    break;
                }
            }

            prez.Absenti = string.Empty;

            foreach (var absent in _mainwindowviewModel._absenti)
            {
                prez.Absenti += absent + "\n";
            }

            prez.Prezenti += receivedName + "\n";
            bz.SaveChanges();
        }
    }
    else
    {
        await SendResponse(context, "Client doesn't exist", 200);
    }
}
```

Figură 36 COD SERVER HTTP pt 2

```
        context.Response.Close();  
    }  
    catch (Exception ex)  
    {  
        Console.WriteLine($"Error: {ex.Message}");  
        context?.Response.Close();  
    }  
}  
  
4 references  
private async Task SendResponse(HttpListenerContext context, string message, int statusCode)  
{  
    byte[] buffer = Encoding.UTF8.GetBytes(message);  
    context.Response.ContentType = "text/plain";  
    context.Response.ContentLength64 = buffer.Length;  
    context.Response.StatusCode = statusCode;  
    await context.Response.OutputStream.WriteAsync(buffer, 0, buffer.Length);  
}  
  
0 references  
public void UnlockClient(string deviceId)  
{  
    _lockedClients.Remove(deviceId);  
}  
  
0 references  
public void UnlockAll()  
{  
    _lockedClients.Clear();  
}  
  
1 reference  
public void Start()  
{  
    _listener.Prefixes.Add("http://localhost:8000/server/");  
    _listener.Start();  
    _isRunning = true;  
    Task.Run(() => HandleRequest());  
}  
  
0 references  
public void Stop()  
{  
    _isRunning = false;  
    _listener.Stop();  
}
```

Figură 37 COD SERVER HTTP pt3

4.2.3 IMPLEMENTAREA COMENZI DE ANALIZA AI

Clasa AnalizaAiCommand este responsabilă pentru gestionarea procesului de analiză a unui videoclip utilizând funcționalitățile oferite de biblioteca CSnakes. Această comandă este conectată la clasa AnalizaAiViewModel, care are rolul de a afișa rezultatele analizei în interfața aplicației. Pentru efectuarea procesului de analiză, clasa utilizează funcționalitățile puse la dispoziție de CSnakesInitialization, aceasta reprezentând punctul de legătură dintre aplicația WPF și modulele Python folosite pentru analiza video.

Integrarea cu biblioteca CSnakes se realizează prin intermediul metodei AnalyzeFight, disponibilă în obiectul CSnakesInitialization._module. Această metodă primește ca parametri calea către videoclipul selectat de utilizator, numele sportivului analizat, un identificator pentru adversar și calea unde vor fi salvate rezultatele procesării. În urma executării analizei, metoda returnează un obiect în format JSON care conține informații detaliate despre luptă, precum numărul de lovituri reușite, lovituri ratate și alte statistici relevante generate de sistemul de inteligență artificială.

Procesul începe prin selectarea unui videoclip de către utilizator cu ajutorul unui dialog de tip OpenFileDialog. Acest mecanism permite alegerea fișierului video ce urmează să fie analizat. În cazul în care utilizatorul închide fereastra fără a selecta un fișier, operația este anulată și procesul de analiză nu mai continuă.

După selectarea videoclipului, aplicația creează automat o locație destinată stocării rezultatelor. În directorul principal al aplicației este generat un folder numit VideoAi, iar în

interiorul acestuia sunt salvate fișierele rezultate în urma analizei. Pentru a evita suprascrierea datelor existente, numele fișierelor este generat pe baza numelui utilizatorului și a altor informații care asigură unicitatea acestora.

În etapa următoare este apelată metoda `AnalyzeFight`, care rulează asincron pentru a evita blocarea interfeței grafice în timpul procesării videoclipului. În această fază, videoclipul este transmis către modelele de inteligență artificială care identifică și urmăresc sportivii, detectează loviturile și generează statisticile necesare. La finalul procesului, rezultatele sunt returnate sub forma unui document JSON.

Ultima etapă constă în procesarea rezultatelor obținute. Datele JSON sunt deserializate într-un obiect de tip `FightResults`, ceea ce permite accesarea facilă a informațiilor generate de analiză. Dacă analiza este validă și conține date despre utilizatorul selectat, statisticile relevante sunt extrase și salvate în baza de date prin intermediul clasei `Bazadateconnect`. Astfel, rezultatele pot fi consultate ulterior în aplicație, contribuind la monitorizarea performanței sportivului și la analiza evoluției sale în timp.

În imaginile de mai jos este codul pentru aceasta comandă.

```
namespace Modern.ViewModel
{
    2 references
    public class AnalizaAICommand : CommandBase
    {
        OameniViewModel _omviewmodel;

        1 reference
        public AnalizaAICommand(OameniViewModel oameniviewmodel)
        {
            _omviewmodel = oameniviewmodel;
        }

        1 reference
        public override async void Execute(object parameter)
        {
            var dialog = new OpenFileDialog();
            dialog.Filter = "Video Files|*.mp4;*.avi;*.mov;*.mkv;*.wmv";

            bool? result = dialog.ShowDialog();

            if (result != true)
                return;

            string selectedVideoPath = dialog.FileName;
            string extension = Path.GetExtension(selectedVideoPath);

            string destinationFolder =
                Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "VideoAi");

            Directory.CreateDirectory(destinationFolder);

            string destinationPath =
                Path.Combine(destinationFolder, _omviewmodel.nume + extension);

            try
            {
                string json = await Task.Run(() =>
                {
                    return CSnakesInitialization._module.AnalyzeFight(
                        selectedVideoPath,
                        _omviewmodel.nume,
                        "inamic",
                        destinationPath
                    );
                });

                FightResults? fightResults =
                    JsonSerializer.Deserialize<FightResults>(json);

                if (fightResults == null)
                {
                    MessageBox.Show("Failed to parse AI results.");
                    return;
                }

                if (!fightResults.Results.ContainsKey(_omviewmodel.nume))
                {
                    MessageBox.Show("Fighter not found in results.");
                }
            }
        }
    }
}
```

Figură 38 COD ANALIZA AI pt 1

```

        if (!fightResults.Results.ContainsKey(_omviewmodel.nume))
        {
            MessageBox.Show("Fighter not found in results.");
            return;
        }

        FighterStats myStats =
            fightResults.Results[_omviewmodel.nume];

        using (Bazadateconnect bz = new Bazadateconnect())
        {
            Oameni om =
                bz.Oameni.FirstOrDefault(o => o.Name == _omviewmodel.nume);

            if (om != null)
            {
                om.PumniNimeritiCap += myStats.HeadLanded;
                om.PumniNimeritiCorp += myStats.BodyLanded;
                om.PumniIncasatiCap += myStats.HeadReceived;
                om.PumniIncasatiCorp += myStats.BodyReceived;
                om.PumniRatati += myStats.TotalMissed;

                bz.SaveChanges();
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error: " + ex.Message);
    }
    finally
    {
        MessageBox.Show("AI analysis completed. Results have been saved to the database.");
    }
}
}

```

Figură 39 COD ANALIZA AI pt 2

4.2.4 ADAUGARE SPORTIV

```

AdaugareOmViewModel _adaugareomviewmodel;

1 reference
public AdaugareOmCommand(AdaugareOmViewModel _adaugareomviewmodel)
{
    _adaugareomviewmodel = _adaugareomviewmodel;
}

1 reference
public override void Execute(object parameter)
{
    if (parameter is Window wind)
    {
        OameniViewModel om = new OameniViewModel(_adaugareomviewmodel.Username, DateTime.Now, _adaugareomviewmodel._mainwindowviewmodel._mainwindowviewmodel.elight == true)
        {
            if (_adaugareomviewmodel._mainwindowviewmodel.elight == true)
            {
                om.Gri = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#9802C9"));
                om.Galben = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#285781"));
                om.Rosu = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#4F959D"));
                om.Negru = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#F6F8D5"));
                if (DateTime.Now.Date > om.data.AddMonths(1))
                {
                    om.expirat = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#285781"));
                }
                else
                {
                    om.expirat = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#F6F8D5"));
                }
            }
            else
            {
                om.Gri = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#343131"));
                om.Galben = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#D8A25E"));
                om.Rosu = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#A84747"));
                om.Negru = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#242222"));
                if (DateTime.Now.Date > om.data.AddMonths(1))
                {
                    om.expirat = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#A84747"));
                }
                else
                {
                    om.expirat = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#D8A25E"));
                }
            }
        }

        Oameni bzom = new Oameni()
        {
            Name = om.nume,
            Abonament = om.data,
            ArePoza = false,
            PumniNimeritiCap = 0,
            PumniNimeritiCorp = 0,
            PumniIncasatiCap = 0,
            PumniIncasatiCorp = 0,
            PumniRatati = 0,
            ProcentajPrezenta = 0,
            NumarVictorii = 0,
        }
    }
}

```

Figură 40 COD ADAUGARE SPORTIV pt 1

```
};  
using (Bazadateconnect bazadate = new Bazadateconnect())  
{  
    bazadate.Oameni.Add(bzom);  
  
    Tranzactie tranz = new Tranzactie()  
    {  
        Data = DateTime.Now,  
        Suma = _adaugareomviewmodel._mainwindowviewmodel._pretabonament,  
        EVenit = true,  
        Descriere = $"Un nou membru a fost inscris : {om.nume} Abonament:{_adaugareomviewmodel._mainwindowviewmodel._pretabonament}"  
    };  
  
    bazadate.Tranzactii.Add(tranz);  
    bazadate.SaveChanges();  
}  
  
_adaugareomviewmodel._mainwindowviewmodel._oameni.Add(om);  
  
wind.Close();  
}
```

Figură 41 COD ADAUGARE SPORTIV pt2

Clasa `AdaugareOmCommand` reprezintă o comandă utilizată în arhitectura MVVM pentru adăugarea unui nou membru în aplicație. Aceasta primește prin constructor o referință către `AdaugareOmViewModel`, de unde preia informațiile introduse de utilizator. În momentul executării comenzii, se verifică dacă parametrul primit este o fereastră validă, după care este creat un nou obiect de tip `OameniViewModel`, care conține numele membrului și data curentă ce marchează începutul abonamentului. În funcție de tema selectată în aplicație (light sau dark), sunt configurate diferite culori pentru elementele vizuale asociate membrului, inclusiv culoarea care indică starea abonamentului. Dacă data curentă depășește perioada de o lună de la activarea abonamentului, membrul este marcat ca având abonamentul expirat prin schimbarea culorii indicatorului vizual.

După configurarea obiectului utilizat în interfață, este creat un obiect de tip `Oameni`, care reprezintă entitatea ce va fi salvată în baza de date. Acesta este inițializat cu valorile introduse de utilizator și cu statistici setate implicit la zero, deoarece membrul nu are încă activitate înregistrată. Prin intermediul contextului `Bazadateconnect`, noul membru este adăugat în tabela corespunzătoare din baza de date. În aceeași tranzacție este creat și un obiect de tip `Tranzactie`, care înregistrează plata abonamentului ca un venit pentru sală, stocând informații precum data operației, suma încasată și o descriere a tranzacției. După adăugarea ambelor entități, modificările sunt salvate în baza de date prin apelul metodei `SaveChanges()`. În final, noul membru este adăugat și în colecția de membri afișată în interfața aplicației, iar fereastra de adăugare este închisă, semnalând finalizarea cu succes a procesului de înregistrare.

4.2.5 ADAUGARE SPORTIV

Clasa `AdaugaArticolCommand` este responsabilă pentru procesul de adăugare a unui nou articol în inventarul aplicației, fiind implementată conform modelului arhitectural MVVM. Prin intermediul constructorului, aceasta primește referințe către `AdaugareArticolViewModel`, care conține datele introduse de utilizator, și către `InventarViewModel`, care gestionează lista articolelor afișate în interfață. În momentul executării comenzii, se verifică dacă parametrul primit este o fereastră validă, după care este creat un nou obiect de tip `Inventar`, care reprezintă entitatea ce va fi stocată în baza de date. Acest obiect este inițializat cu informațiile introduse de utilizator, precum denumirea articolului, prețul de cumpărare, prețul de vânzare, cantitatea disponibilă în stoc și informația care indică dacă articolul reprezintă un echipament. Ulterior, este creat un obiect de tip `ArticolViewModel`, utilizat pentru reprezentarea articolului în interfața grafică a aplicației. Dacă utilizatorul a selectat o imagine pentru articol, aceasta este asociată obiectului din interfață, iar câmpul `ArePoza` este setat la valoarea adevărată; în caz contrar, articolul este marcat ca neavând imagine asociată. După configurarea tuturor proprietăților, articolul este adăugat în colecția de articole gestionată de `InventarViewModel`, astfel încât să fie afișat imediat în interfață. În continuare, prin intermediul contextului `Bazadateconnect`, noul articol este adăugat în baza de date și salvat folosind metoda `SaveChanges()`, asigurând persistența informațiilor. La final, fereastra de adăugare este închisă, indicând faptul că operația s-a încheiat cu succes. În imaginea de mai jos apare codul al acestei comenzi.

```
namespace Modern.ViewModel
{
    2 references
    public class AdaugaArticolCommand : CommandBase
    {
        InventarViewModel _inventarviewModel;
        AdaugareArticolViewModel _articol;

        1 reference
        public AdaugaArticolCommand(AdaugareArticolViewModel articol, InventarViewModel inventarviewModel)
        {
            this._articol = articol;
            this._inventarviewModel = inventarviewModel;
        }

        1 reference
        public override void Execute(object parameter)
        {
            if (parameter is Window wind)
            {
                Inventar art = new()
                {
                    Denumire = _articol.Denumire,
                    PretCumparare = _articol.PretCumparare,
                    PretVanzare = _articol.Pretvanzare,
                    Stoc = _articol.Stoc,
                    EsteEchipament = _articol.EsteEchipament,
                };

                ArticolViewModel articol = new ArticolViewModel(art.Denumire, art.PretVanzare, art.Stoc, art.PretCumparare, _inventarviewModel);

                if (_articol.ArticolImage != null)
                {
                    articol.ProfileImage = _articol.ArticolImage;
                    art.ArePoza = true;
                }
                else
                {
                    art.ArePoza = false;
                }

                _inventarviewModel._articole.Add(articol);

                using (Bazadateconnect bz = new Bazadateconnect())
                {
                    bz.Articole.Add(art);
                    bz.SaveChanges();
                }

                wind.Close();
            }
        }
    }
}
```

Figură 42 COD ADAUGARE ARTICOL

4.2.6 REINOIRE ABONAMENT

Clasa `RenoireAbonamentCommand` este responsabilă pentru reînnoirea abonamentului unui membru. La executarea comenzii, data abonamentului este actualizată cu data curentă, iar indicatorul vizual care semnalează expirarea abonamentului este modificat în funcție de tema activă a aplicației. Ulterior, prin intermediul clasei `Bazadateconnect`, este identificat membrul corespunzător în baza de date și este actualizată data abonamentului acestuia. În același timp, este creată o nouă înregistrare de tip `Tranzactie`, care consemnează plata aferentă reînnoirii abonamentului, incluzând descrierea operației, data și suma încasată. La final, toate modificările sunt salvate în baza de date prin apelul metodei `SaveChanges()`. În imaginea de mai jos apare codul acestei comenzi.

```
namespace Modern.ViewModel
{
    2 references
    class RenoireAbonamentCommand : Commandbase
    {
        OameniViewModel _omviewmodel;

        1 reference
        public RenoireAbonamentCommand(OameniViewModel oameniviewmodel)
        {
            _omviewmodel = oameniviewmodel;
        }
        1 reference
        public override void Execute(object parameter)
        {
            _omviewmodel.data = DateTime.Now;
            if (_omviewmodel._mainviewmodel.elight == true)
            {
                _omviewmodel.expirat = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#F6F8D5"));
            }
            else
            {
                _omviewmodel.expirat = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#D8A25E"));
            }

            using (Bazadateconnect bz = new Bazadateconnect())
            {
                Oameni om = bz.Oameni.FirstOrDefault(o => o.Name == _omviewmodel.nume);

                if (om != null)
                {
                    om.Abonament = DateTime.Now;
                }

                Tranzactie tran = new Tranzactie
                {
                    Descriere = $"Renoire abonament pentru {_omviewmodel.nume}",
                    Data = DateTime.Now,
                    Suma = _omviewmodel._mainviewmodel._pretabonament,
                    EVenit = true,
                };

                bz.Tranzactii.Add(tran);

                bz.SaveChanges();
            }
        }
    }
}
```

Figură 43 COD REINOIRE ABONAMENT

4.2.7 SCHIMBARE TEMATICA

Clasa SchimbareTemaCommand gestionează schimbarea temei vizuale a aplicației între modul light și dark. La executarea comenzii, aceasta verifică valoarea variabilei `elight` din `MainWindowViewModel` pentru a determina tema activă. În cazul trecerii la tema light, sunt actualizate culorile principale ale interfeței, iar aceeași schimbare este aplicată tuturor membrilor din lista `_oameni`, inclusiv culorile asociate stării abonamentului. Dacă abonamentul unui membru este expirat, culoarea indicatorului este modificată corespunzător. În cazul schimbării la tema dark, procesul este similar, fiind aplicate alte valori de culoare pentru interfață și pentru lista de membri. La final, variabila `elight` este actualizată pentru a reține tema curentă selectată de utilizator. În imaginea de mai jos avem codul comenzii.

```
1 reference
public override void Execute(object parameter)
{
    if (_mainwindowmodel.elight == false)
    {
        _mainwindowmodel.Gri = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#98D2C0"));
        _mainwindowmodel.Galben = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#205781"));
        _mainwindowmodel.Rosu = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#4F959D"));
        _mainwindowmodel.Negru = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#F6F8D5"));

        _mainwindowmodel.elight = true;

        foreach (var a in _mainwindowmodel._oameni)
        {
            a.Rosu = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#4F959D"));
            a.Negru = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#F6F8D5"));
            a.Galben = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#205781"));
            a.Gri = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#98D2C0"));
            if (DateOnly.FromDateTime(DateTime.Today) > a.data.AddMonths(1))
            {
                a.expirat = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#205781"));
            }
            else
            {
                a.expirat = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#F6F8D5"));
            }
        }
    }
    else
    {
        _mainwindowmodel.Gri = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#343131"));
        _mainwindowmodel.Galben = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#D8A25E"));
        _mainwindowmodel.Rosu = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#A04747"));
        _mainwindowmodel.Negru = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#242222"));

        foreach (var a in _mainwindowmodel._oameni)
        {
            a.Gri = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#343131"));
            a.Galben = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#D8A25E"));
            a.Rosu = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#A04747"));
            a.Negru = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#242222"));
            if (DateOnly.FromDateTime(DateTime.Today) > a.data.AddMonths(1))
            {
                a.expirat = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#A04747"));
            }
            else
            {
                a.expirat = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#D8A25E"));
            }
        }

        _mainwindowmodel.elight = false;
    }
}
```

Figură 44 COD SCHIMBARE TEMATICA

4.2.8 ADAUGARE POZA

Clasa `AdaugarePozaCommand` este responsabilă pentru adăugarea unei imagini asociate unui membru din aplicație. La executarea comenzii, este deschis un dialog de selectare a fișierelor cu ajutorul clasei `OpenFileDialog`, fiind acceptate formate de imagine precum JPG, JPEG, PNG și JFIF. După alegerea unei imagini de către utilizator, fișierul este copiat în folderul `Imagini` din directorul aplicației, folosind numele sportivului pentru identificarea acestuia. Imaginea copiată este apoi încărcată în interfață prin intermediul unui obiect `BitmapImage`, astfel încât să fie afișată utilizatorului. În final, prin intermediul clasei `Bazadateconnect`, membrul corespunzător este identificat în baza de date, iar proprietatea `ArePoza` este actualizată pentru a indica faptul că acesta are o imagine asociată, modificările fiind salvate prin metoda `SaveChanges()`. În imaginea de mai jos avem codul comenzii.

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Media.Imaging;
using Microsoft.Win32;
using Modern.Model;

namespace Modern.ViewModel
{
    2 references
    public class AdaugarePozaCommand : CommandBase
    {
        DetaliSportiviViewModel _detalisportivviewmodel;

        1 reference
        public AdaugarePozaCommand(DetaliSportiviViewModel detali) {
            this._detalisportivviewmodel = detali;
        }

        1 reference
        public override void Execute(object parameter)
        {
            var dialog = new OpenFileDialog();
            dialog.Filter = "Image Files|*.jpg;*.jpeg;*.png;*.jfif";

            bool? result = dialog.ShowDialog();
            if (result == true)
            {
                string selectedImagePath = dialog.FileName;
                string extension = Path.GetExtension(selectedImagePath);
                string destinationFolder = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "Imagini");
                Directory.CreateDirectory(destinationFolder);
                string destinationPath = Path.Combine(destinationFolder, _detalisportivviewmodel._oameni.nume + extension);

                File.Copy(selectedImagePath, destinationPath, true);

                _detalisportivviewmodel.ProfileImage = new BitmapImage(new Uri(destinationPath));
            }

            using (Bazadateconnect bz = new Bazadateconnect())
            {
                Oameni om = bz.Oameni.FirstOrDefault(o => o.Name == _detalisportivviewmodel._oameni.nume);

                om.ArePoza = true;
                bz.SaveChanges();
            }
        }
    }
}
```

Figură 45 COD ADAUGARE POZA

4.2.9 IMPLEMENTARE VIEWMODEL

În imaginea de mai jos este prezentat un exemplu de ViewModel utilizat în aplicație, mai exact `MainWindowViewModel`, care reprezintă ViewModel-ul principal al aplicației. Acesta are rolul de a realiza legătura dintre interfața grafică (View) și partea de logică a aplicației, conform arhitecturii MVVM (Model-View-ViewModel). ViewModel-ul gestionează

datele afișate, comenzile disponibile și comunicarea cu modelul, astfel încât interfața să nu conțină logică de procesare.

Un ViewModel este format din proprietăți care sunt conectate cu elementele din interfață prin mecanismul de data binding oferit de WPF. Acest mecanism permite ca valorile din ViewModel să fie afișate automat în View și ca modificările realizate în aplicație să fie transmise între cele două componente. De exemplu, proprietatea Filtru din MainWindowViewModel este legată de o casetă de căutare din interfață. Când utilizatorul introduce un text, valoarea este transmisă către ViewModel, iar prin metoda OnPropertyChanged interfața este notificată și lista membrilor este actualizată automat.

ViewModel-ul gestionează și colecțiile de date afișate în aplicație. În cazul de față, lista membrilor este stocată într-un ObservableCollection de tip OameniViewModel, care permite actualizarea automată a interfeței atunci când sunt adăugate sau modificate elemente. La pornirea aplicației, metoda IaOameniDinBD() extrage informațiile despre membri din baza de date și creează obiecte de tip ViewModel pentru fiecare membru, acestea fiind apoi afișate în interfață.

Un alt element important al unui ViewModel este reprezentat de comenzile ICommand. În arhitectura MVVM, butoanele din interfață sunt conectate la comenzi din ViewModel, nu direct la metode din fereastră. De exemplu, comenzile DeschidereAdgOm, DeschidereInventar, DeschidereAnalizaAi sau DeschidereFinante sunt asociate butoanelor din meniul principal, iar atunci când utilizatorul apasă un buton, comanda corespunzătoare este executată și realizează acțiunea dorită, precum deschiderea unei noi pagini.

În cazul aplicației dezvoltate, MainWindowViewModel are un rol central deoarece controlează funcționalități importante precum navigarea între pagini, schimbarea temei vizuale, filtrarea membrilor, gestionarea prezenței și pornirea serverului HTTP pentru comunicarea cu alte componente. Prin separarea logicii în ViewModel și păstrarea interfeței independente, aplicația devine mai organizată, mai ușor de modificat și mai simplu de extins. În imaginile de mai jos este întregul cod al acestui viewModel.

```
namespace Modern.ViewModel
{
    36 references
    public class MainWindowViewModel:ViewModelBase
    {
        public ObservableCollection<OameniViewModel> _oameni;
        3 references
        public ICollectionView Oameni { get; }

        private string _filtrare=string.Empty;

        2 references
        public string Filtru
        {
            get { return _filtrare; }

            set
            {
                _filtrare = value;
                OnPropertyChanged(nameof(Filtru));
                Oameni.Refresh();
            }
        }

        private Brush _gri= new SolidColorBrush((Color)ColorConverter.ConvertFromString("#343131"));
        3 references
        public Brush Gri
        {
            get => _gri;
            set
            {
                _gri = value;
                OnPropertyChanged(nameof(Gri));
            }
        }

        private Brush _galben= new SolidColorBrush((Color)ColorConverter.ConvertFromString("#D8A25E"));
        3 references
        public Brush Galben
        {
            get => _galben;
            set
            {
                _galben = value;
                OnPropertyChanged(nameof(Galben));
            }
        }

        private Brush _negru = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#242222"));
        3 references
        public Brush Negru
        {
            get => _negru;
            set
            {
                _negru = value;
                OnPropertyChanged(nameof(Negru));
            }
        }
    }
}
```

Figură 46 COD VIEWMODEL pt 1

```
private Brush _rosu = new SolidColorBrush((Color)ColorConverter.ConvertFromString("#A04747"));
3 references
public Brush Rosu
{
    get => _rosu;
    set
    {
        _rosu = value;
        OnPropertyChanged(nameof(Rosu));
    }
}

1 reference
public List<OameniViewModel> _prezenti { get; set; }

1 reference
public ICommand DeschidereAdgOm { get; }
1 reference
public ICommand SchimbareTema { get; }

1 reference
public ICommand DeschidereAdgAdmin { get; }

public ICommand DeschidereStergereAdmin { get; }

1 reference
public ICommand DeschidereAnalizaAi { get; }

public ICommand DeschidereInventar { get; }

public ICommand DeschiderePrezenta { get; }

public ICommand DeschidereFinante { get; }

public bool elight = false;

public int _pretabonament;

2 references
public string PretAbonament
{
    get => _pretabonament == 0 ? "" : _pretabonament.ToString();

    set
    {
        if (string.IsNullOrEmpty(value))
        {
            _pretabonament = 0;
            OnPropertyChanged(nameof(PretAbonament));
            return;
        }

        if (int.TryParse(value, out int pret))
        {
            _pretabonament = pret;
            OnPropertyChanged(nameof(PretAbonament));
        }
    }
}
```

Figură 47 COD VIEWMODEL pt 2

```

5 reference
public List<string> _absenti { get; set; }

HttpListener _listener;
HttpServer _server;
1 reference
public MainWindowViewModel()
{
    _oameni = new ObservableCollection<OameniViewModel>();
    _prezenti = new List<OameniViewModel>();
    _absenti = new List<string>();

    IaOameniDinBD();
    CrearePrezentaAzi();

    Oameni = CollectionViewSource.GetDefaultView(_oameni);
    Oameni.Filter = FiltrareOameni;
    DeschidereAdgOm = new DeschidereAdaugareOmCommand(this);
    SchimbareTema = new SchimbareTemaCommand(this);
    DeschidereAdgAdmin = new DeschidereAdaugareAdminCommand(this);
    DeschidereStergereAdmin = new DeschidereStergereAdminCommand(this);
    DeschidereAnalizaAi = new DeschidereAnalizaAiCommand(this);
    DeschidereInventar = new DeschidereInventarCommand(this);
    DeschiderePrezenta = new DeschiderePrezentaCommand(this);
    DeschidereFinante = new DeschidereFinanteCommand(this);

    _listener = new HttpListener();
    _server = new HttpServer(_listener, this);
    _server.Start();
}

1 reference
private bool FiltrareOameni(object obj)
{
    if (obj is OameniViewModel oameni)
    {
        return oameni.nume.Contains(Filtru, StringComparison.InvariantCultureIgnoreCase);
    }
    return false;
}

1 reference
public void CrearePrezentaAzi()
{
    using (Bazadateconnect bz = new Bazadateconnect())
    {
        string absenti = string.Empty;

        foreach (var om in _oameni)
        {
            _absenti.Add(om.nume);
            absenti += om.nume + "\n";
        }

        var prezenta = new Prezenta
        {
            Data = DateOnly.FromDateTime(DateTime.Today),
            Prezenti = string.Empty,
            Absenti = absenti
        };
    }
}
    
```

Figură 48 COD VIEWMODEL pt 3

```

        Data = DateOnly.FromDateTime(DateTime.Today),
        Prezenti = string.Empty,
        Absenti = absenti
    };

    if (!bz.Prezenti.Any())
    {
        bz.Prezenti.Add(prezenta);
        bz.SaveChanges();
        return;
    }

    foreach (var prez in bz.Prezenti)
    {
        if (prez.Data == DateOnly.FromDateTime(DateTime.Today))
        {
            return;
        }
    }
    bz.Prezenti.Add(prezenta);
    bz.SaveChanges();
}

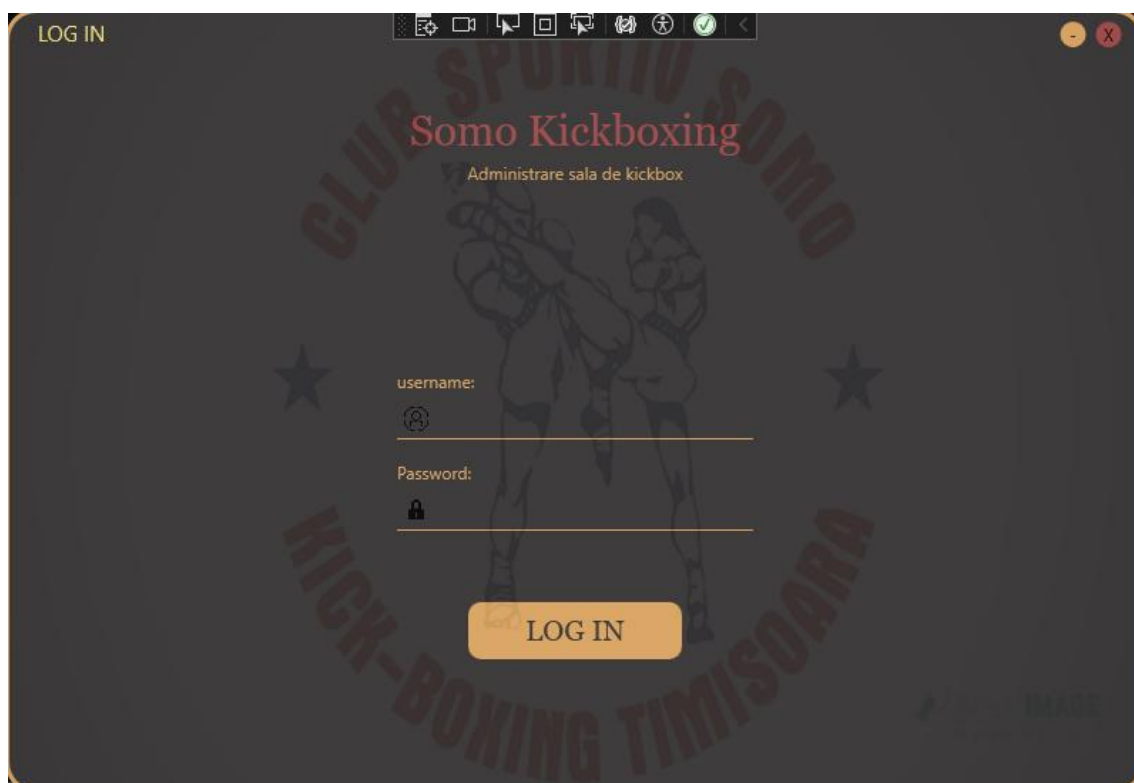
1 reference
public void IaOameniDinBD()
{
    using (Bazadateconnect bz = new Bazadateconnect())
    {
        foreach (var om in bz.Oameni)
        {
            var nou = new OameniViewModel(om.Name, om.Abonament, this);
            _oameni.Add(nou);
        }
    }
}
}
    
```

Figură 49 COD VIEWMODEL pt 4

4.2.10 INTERFATA GRAFICA

În acest subcapitol voi prezenta principalele pagini ale interfeței aplicației WPF, precum și rolul fiecăreia în funcționarea sistemului. Interfața a fost proiectată astfel încât să fie intuitivă și ușor de utilizat, oferind acces rapid la toate funcționalitățile importante ale aplicației. Fiecare pagină are un scop bine definit și permite utilizatorului să interacționeze eficient cu datele și operațiile disponibile. În continuare vor fi descrise principalele componente ale interfeței, modul în care acestea sunt organizate și funcționalitățile pe care le oferă utilizatorilor.

4.2.10.1 LOGIN



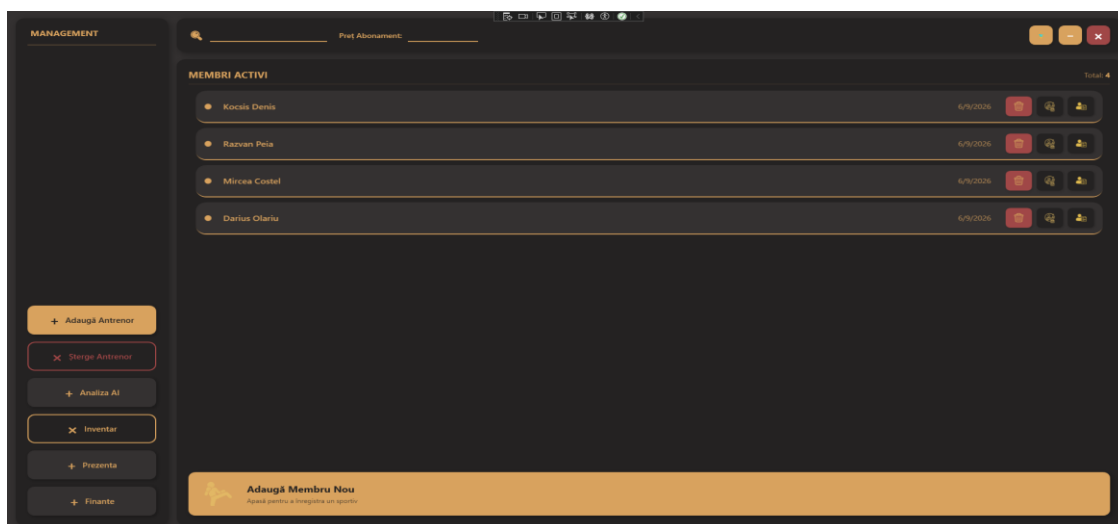
Figură 50 PAGINA LOGIN

Pagina de login permite autentificarea administratorului în aplicație prin introducerea numelui de utilizator și a parolei. Interfața are un design modern, include butoane pentru conectare, minimizare și închidere, iar în cazul unor date incorecte afișează un mesaj de eroare. Implementarea respectă modelul MVVM, separând interfața de logica aplicației.

4.2.10.2 PAGINA PRINCIPALA

Pagina principală reprezintă centrul de control al aplicației, de unde pot fi gestionate toate funcționalitățile importante ale sistemului. Prin intermediul meniului lateral, utilizatorul poate accesa modulele pentru administrarea antrenorilor, gestionarea membrilor, inventar,

prezență, finanțe și analiza video cu inteligență artificială. Tot din această pagină se pot vizualiza membrii activi, căuta sportivi după diferite criterii, reînnoi abonamente, șterge membri sau accesa informații detaliate despre aceștia. Astfel, pagina principală oferă acces rapid și centralizat la toate operațiile necesare administrării sălii.



Figură 51 PAGINA PRINCIPALA

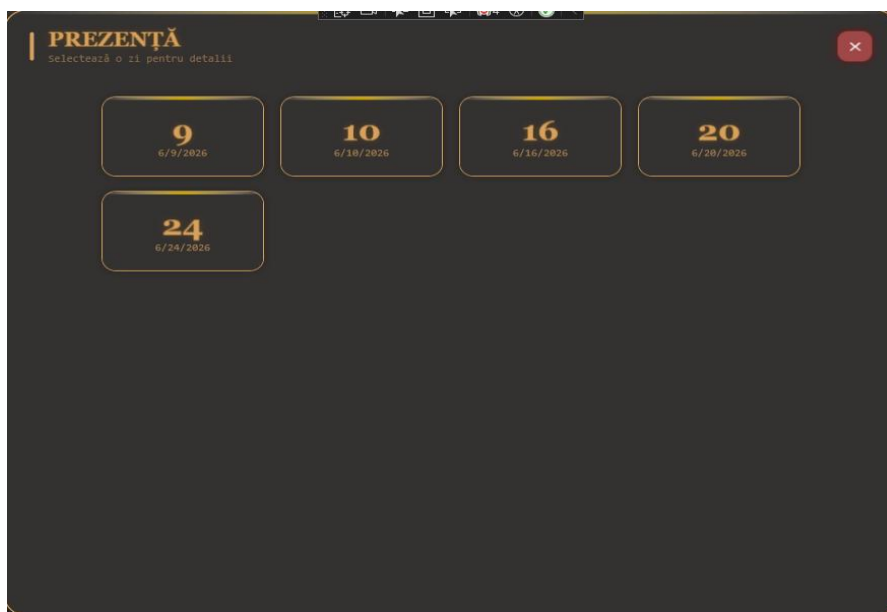
4.2.10.3 DETALI SPORTIV



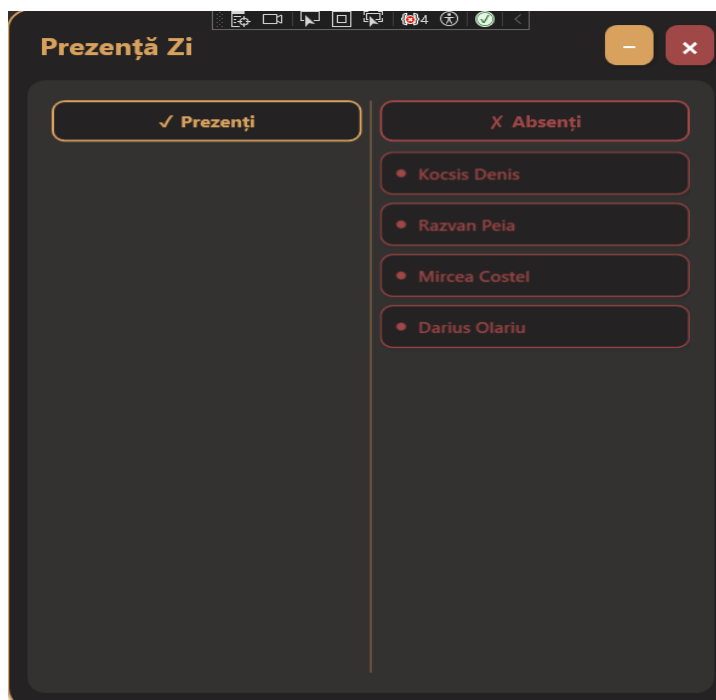
Figură 52 PAGINA DETALI SPORTIV

În aceasă pagină sunt afișate informațiile detaliate ale fiecărui sportiv. Utilizatorul poate adăuga și gestiona punctele tari și punctele slabe ale sportivului, oferind o evaluare clară a performanțelor și aspectelor care necesită îmbunătățire. De asemenea, poate fi selectat nivelul de experiență al sportivului, iar în partea de statistici sunt prezentate procentajul de prezență la antrenamente și diverse grafice care oferă o imagine de ansamblu asupra activității și progresului acestuia. Astfel, pagina permite monitorizarea și analiza detaliată a fiecărui membru al sălii.

4.2.10.4 PREZENTE



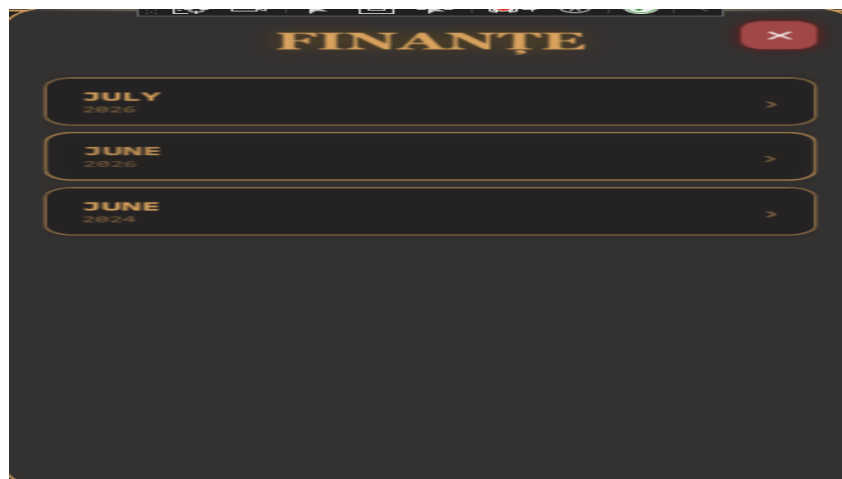
Figură 53 PAGINA PREZENTE pt 1



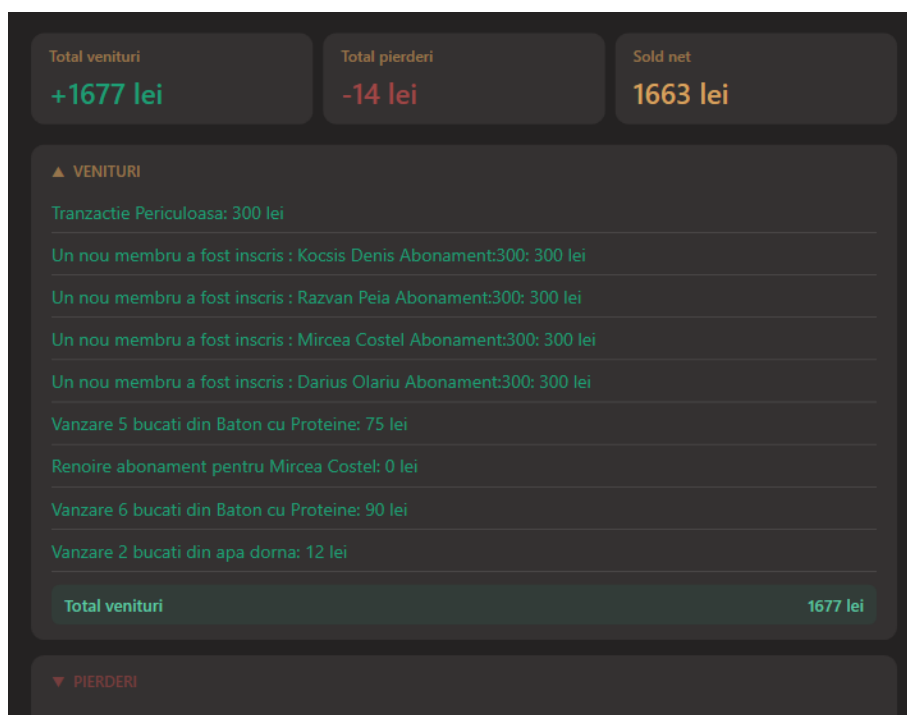
Figură 54 PAGINA PREZENTE pt 2

În aceste două pagini putem vizualiza evidența prezențelor, unde sunt afișate zilele în care s-a realizat verificarea prezenței și persoanele care au participat în fiecare zi. De asemenea, aplicația permite identificarea rapidă a membrilor prezenți și absenți, oferind o imagine clară asupra activității și frecvenței acestora. Aceste informații ajută la monitorizarea participării și la gestionarea mai eficientă a datelor despre prezență.

4.2.10.5 FINANTE



Figură 55 PAGINA FINANTE pt 1

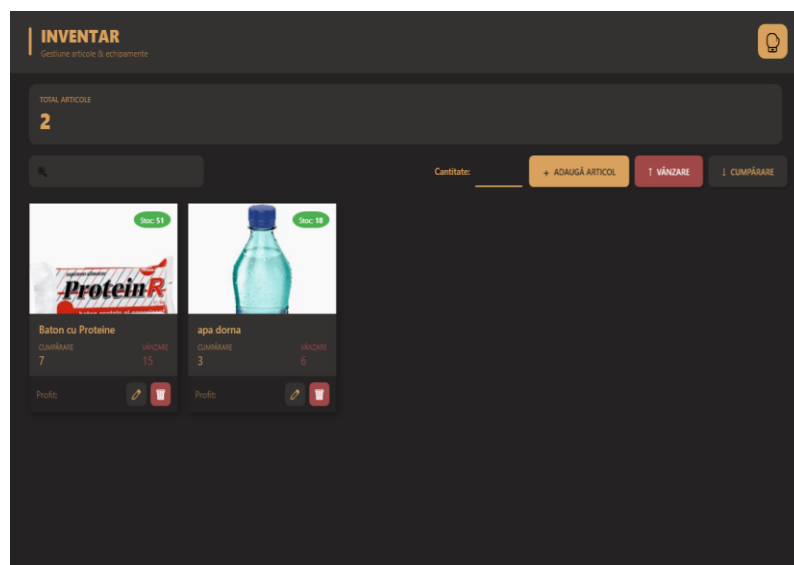


Figură 56 PAGINA FINANTE pt 2

În aceste două imagini sunt prezentate paginile dedicate gestionării finanțelor. În prima pagină, utilizatorul poate selecta luna pentru care dorește să consulte informațiile financiare. După alegerea lunii, în cea de-a doua pagină sunt afișate toate veniturile și cheltuielile înregistrate în perioada respectivă. Astfel, utilizatorul poate analiza rapid situația financiară a lunii selectate, având o evidență clară a încasărilor și a cheltuielilor efectuate.

4.2.10.6 INVENTAR

Pagina de inventar este destinată gestionării articolelor și echipamentelor disponibile în cadrul aplicației. În partea superioară este afișat numărul total de articole existente în inventar, iar utilizatorul poate căuta rapid un produs folosind bara de căutare. Tot aici poate introduce o cantitate și poate efectua operații precum adăugarea unui articol nou, înregistrarea unei vânzări sau a unei cumpărări. În partea principală a paginii sunt afișate toate articolele sub forma unor carduri care conțin imaginea produsului, denumirea, stocul disponibil, prețul de cumpărare și prețul de vânzare. Pentru fiecare articol există opțiuni de editare și ștergere, ceea ce permite administrarea eficientă a inventarului și actualizarea rapidă a informațiilor despre produse.



Figură 57 PAGINA INVENTAR pt 1

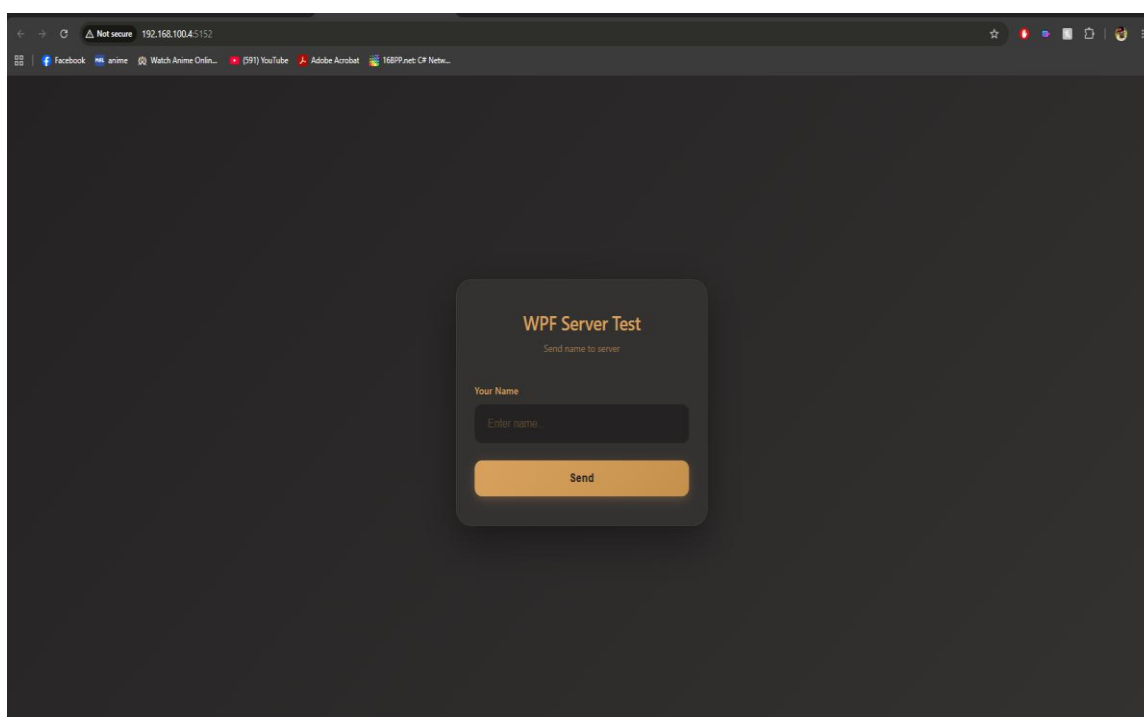
The screenshot shows the 'Adăugare Articol' (Add Item) form. It has a title bar with a close button. The form contains several input fields: 'DENUMIRE ARTICOL' (Article Name), 'STOC DISPONIBIL' (Available Stock), 'PREȚ CUMPĂRARE' (Purchase Price), and 'PREȚ VÂNZARE' (Selling Price). There is a checkbox labeled 'Este echipament sportiv' (Is sports equipment). To the right of the price fields is a large dashed circle labeled 'IMAGINE ARTICOL' (Article Image). At the bottom, there is a large orange button labeled '+ SALVEAZĂ ARTICOL' (Save Article).

Figură 58 PAGINA INVENTAR pt 2

4.3 API

În acest capitol voi prezenta implementarea API-ului utilizat în cadrul aplicației. Vor fi descrise principalele componente ale acestuia, modul în care sunt realizate cererile și răspunsurile dintre aplicație și server, precum și metodele folosite pentru transmiterea și prelucrarea datelor. De asemenea, vor fi explicate funcționalitățile oferite de API, rolul fiecărui endpoint și modul în care acestea contribuie la funcționarea corectă a aplicației. Scopul acestui capitol este de a evidenția procesul de integrare a API-ului și importanța acestuia în comunicarea dintre diferitele componente ale sistemului.

4.3.1 PAGINAWEB



Figură 59 PAGINA WEB API

În cadrul paginii web puse la dispoziție de API, sportivul are posibilitatea de a introduce numele său într-un câmp dedicat căutării. După trimiterea cererii, informația este transmisă către aplicația WPF, care procesează solicitarea și verifică datele existente în propria bază de date. În funcție de rezultatul obținut, aplicația returnează un răspuns către pagina web, unde sunt afișate informații referitoare la valabilitatea abonamentului sportivului, precum data expirării și starea acestuia. Această metodă permite accesarea rapidă a informațiilor despre abonament, utilizând aplicația WPF ca intermediar între interfața web și baza de date.

4.3.2 CONTROLER PRINCIPAL

```
using Microsoft.AspNetCore.Mvc;
using System.Text;
using System.Text.Json;

namespace WebApi.Controllers
{
    [ApiController]
    [Route("api")]
    1 reference
    public class SubscriptionController : ControllerBase
    {
        private readonly HttpClient _httpClient;

        0 references
        public SubscriptionController(IHttpClientFactory httpClientFactory)
        {
            _httpClient = httpClientFactory.CreateClient();
        }

        [HttpPost("check-subscription")]
        0 references
        public async Task<IActionResult> CheckSubscription([FromBody] NameRequest request)
        {
            try
            {
                var json = JsonSerializer.Serialize(new
                {
                    name = request.Name,
                    deviceId = request.DeviceId
                });
                var content = new StringContent(json, Encoding.UTF8, "application/json");

                var response = await _httpClient.PostAsync("http://localhost:8000/server/", content);
                var result = await response.Content.ReadAsStringAsync();

                return Ok(result);
            }
            catch (Exception ex)
            {
                return StatusCode(500, $"Error communicating with WPF server: {ex.Message}");
            }
        }

        1 reference
        public class NameRequest
        {
            1 reference
            public string Name { get; set; }
            1 reference
            public string DeviceId { get; set; }
        }
    }
}
```

Figură 60 COD CONTROLER API

Acest cod definește un controller API numit SubscriptionController pentru o aplicație ASP.NET Core, care expune un endpoint POST la ruta /api/check-subscription. Când acest endpoint este apelat, metoda CheckSubscription primește un obiect cu nume și deviceId din corpul cererii, serializează aceste date în format JSON și le trimite mai departe către un server WPF local, folosind un request HTTP POST la adresa http://localhost:8000/server/. Răspunsul primit de la serverul WPF este returnat direct către clientul care a făcut cererea inițială. Dacă apare o eroare în timpul comunicării, controllerul returnează un răspuns cu codul 500 și un mesaj de eroare. Astfel, acest controller acționează ca un intermediar între aplicația web și serverul WPF, facilitând validarea abonamentului unui utilizator pe baza datelor primite.

5 CONCLUZIE

În urma realizării acestui proiect, a fost dezvoltată o aplicație complexă pentru administrarea unei săli de fitness, utilizând tehnologia WPF pentru construirea unei interfețe moderne, intuitive și ușor de utilizat. Scopul principal al aplicației a fost digitalizarea proceselor de gestionare a sportivilor și optimizarea activităților desfășurate zilnic în cadrul unei săli de fitness. Pe parcursul dezvoltării au fost implementate funcționalități care permit administrarea eficientă a informațiilor și oferă o experiență mai bună atât pentru personalul sălii, cât și pentru sportivi.

Aplicația permite gestionarea informațiilor despre sportivi, abonamente, prezențe și statistici, oferind posibilitatea de a urmări activitatea acestora într-un mod organizat. Prin intermediul interfeței dezvoltate în WPF, utilizatorul poate accesa și modifica rapid datele necesare, iar utilizarea arhitecturii MVVM a contribuit la o structurare mai clară a proiectului și la o întreținere mai ușoară a codului.

Un element important al aplicației îl reprezintă integrarea unui API care permite comunicarea dintre pagina web și aplicația WPF. Prin intermediul acestuia, sportivii pot introduce numele lor într-o pagină web, iar informația este transmisă către aplicația WPF, care verifică datele din propria bază de date și returnează un răspuns privind valabilitatea abonamentului. Această funcționalitate oferă un mod simplu și rapid prin care utilizatorii pot verifica informațiile despre abonament fără a accesa direct aplicația principală.

De asemenea, proiectul include o componentă bazată pe inteligență artificială pentru analiza materialelor video, oferind posibilitatea de a utiliza tehnologii moderne pentru obținerea unor informații suplimentare. Această parte a proiectului evidențiază modul în care AI poate fi integrată într-o aplicație pentru a extinde funcționalitățile acesteia și pentru a automatiza anumite procese.

Pe parcursul dezvoltării, aplicația a atins obiectivele propuse, însă există și posibilități de îmbunătățire. În viitor, aplicația WPF ar putea fi extinsă prin adăugarea unor noi funcționalități, precum mai multe instrumente de analiză, notificări automate, un sistem mai avansat de monitorizare a progresului sportivilor sau alte opțiuni care să faciliteze administrarea sălii. De asemenea, partea de API ar putea fi dezvoltată prin adăugarea unor noi servicii și funcții care să ofere utilizatorilor mai multe posibilități de interacțiune cu sistemul.

Componenta de inteligență artificială ar putea fi îmbunătățită prin utilizarea unor modele AI mai noi și mai complexe, însă acest lucru necesită resurse hardware mai performante pentru rularea acestora. Dacă ar fi fost disponibile componente hardware mai puternice, ar fi fost posibilă testarea și integrarea unor modele AI mai avansate pentru o analiză video mai precisă și mai eficientă.

În concluzie, proiectul realizat reprezintă o aplicație completă care combină tehnologii precum WPF, baze de date, API-uri și inteligență artificială pentru a crea o soluție modernă de administrare a unei săli de fitness. Aplicația îndeplinește funcționalitățile principale propuse, iar prin dezvoltări ulterioare și utilizarea unor resurse hardware mai performante poate deveni un sistem și mai complex și mai eficient.

LISTA FIGURI

Figură 1 MVVM	13
Figură 2 Entity	14
Figură 3 CSnakes	15
Figură 4 SAM	16
Figură 5 YOLO	17
Figură 6 LIVECHARTS	17
Figură 7 VISUAL STUDIO	18
Figură 8 ROBOFLOW	19
Figură 9 CLASA VIEWMODELBASE	20
Figură 10 CLASA COMMANDBASE	21
Figură 11 STRUCTURA FISIERE	21
Figură 12 DIAGRAMA COMUNICARE APLICATII	22
Figură 13 DIAGRAMA STRUCTURA BAZA DE DATE	23
Figură 14 DIAGRAMA ACTIVITATE SERVER HTTP	25
Figură 15 SCHEMA FUNCTIONAL APLICATIE	26
Figură 16 DIAGRAMA ACTIVITATE INITIALIZARE PYTHON	28
Figură 17 COLECTARE DATE	31
Figură 18 EXEMPLU COD ROBOFLOW	32
Figură 19 REZULTATE DUPA EXTRAGERE ROBOFLOW	32
Figură 20 COD ANTRENARE AI	33
Figură 21 OVERLAP MASCA AI	35
Figură 22 AFLAREA CAPULUI AI	35
Figură 23 OVERLAP CUTI AI	36
Figură 24 CENTRUL CUTII AI	36
Figură 25 SELECTARE LUPTATORI AI	37
Figură 26 CLASA CSNAKESINITIALIZATION	38
Figură 27 CLASA RUNPIPCOMMAND	39
Figură 28 INSTALARE PYTHORCH	40
Figură 29 VERIFICARE CUDA	41
Figură 30 REZULTATE AI	42
Figură 31 CONTEXT BAZA DE DATE	43
Figură 32 CREARE BAZA DE DATE	44
Figură 33 COD LOGIN	45
Figură 34 VIEWMODEL LOGIN	46
Figură 35 COD SERVER HTTP pt 1	48
Figură 36 COD SERVER HTTP pt 2	48
Figură 37 COD SERVER HTTP pt3	49
Figură 38 COD ANALIZA AI pt 1	50
Figură 39 COD ANALIZA AI pt 2	51
Figură 40 COD ADAUGARE SPORTIV pt 1	51

Figură 41 COD ADAUGARE SPORTIV pt2	52
Figură 42 COD ADAUGARE ARTICOL	53
Figură 43 COD REINOIRE ABONAMENT	54
Figură 44 COD SCHIMBARE TEMATICA.....	55
Figură 45 COD ADAUGARE POZA	56
Figură 46 COD VIEWMODEL pt 1	58
Figură 47 COD VIEWMODEL pt 2	59
Figură 48 COD VIEWMODEL pt 3	60
Figură 49 COD VIEWMODEL pt 4	60
Figură 50 PAGINA LOGIN	61
Figură 51 PAGINA PRINCIPALA	62
Figură 52 PAGINA DETALI SPORTIV	62
Figură 53 PAGINA PREZENTE pt 1	63
Figură 54 PAGINA PREZENTE pt 2	63
Figură 55 PAGINA FINANTE pt 1	64
Figură 56 PAGINA FINANTE pt 2	64
Figură 57 PAGINA INVENTAR pt 1	65
Figură 58 PAGINA INVENTAR pt 2	65
Figură 59 PAGINA WEB API.....	66
Figură 60 COD CONTROLER API.....	67

BIBLIOGRAFIE

- [1] Wikipedia, „C #,” Wikipedia, [Interactiv]. Available: [https://en.wikipedia.org/wiki/C_Sharp_\(programming_language\)](https://en.wikipedia.org/wiki/C_Sharp_(programming_language)). [Accesat 15 5 2026].
- [2] [Interactiv]. Available: [https://en.wikipedia.org/wiki/Python_\(programming_language\)](https://en.wikipedia.org/wiki/Python_(programming_language)).
- [3] [Interactiv]. Available: <https://en.wikipedia.org/wiki/HTTP>.
- [4] [Interactiv]. Available: https://en.wikipedia.org/wiki/Extensible_Application_Markup_Language.
- [5] [Interactiv]. Available: <https://en.wikipedia.org/wiki/JavaScript>.
- [6] [Interactiv]. Available: <https://en.wikipedia.org/wiki/CSS>.
- [7] [Interactiv]. Available: https://www.ibm.com/think/topics/computer-vision?utm_source=chatgpt.com.
- [8] [Interactiv]. Available: https://en.wikipedia.org/wiki/Windows_Presentation_Foundation.
- [9] [Interactiv]. Available: <https://www.geeksforgeeks.org/c-sharp/what-is-entity-framework-in-net-framework/>.
- [10] [Interactiv]. Available: <https://tonybaloney.github.io/CSnakes/v1/>.
- [11] [Interactiv]. Available: https://en.wikipedia.org/wiki/ASP.NET_Core.
- [12] [Interactiv]. Available: <https://ai.meta.com/research/sam2/>.
- [13] [Interactiv]. Available: https://en.wikipedia.org/wiki/You_Only_Look_Once.
- [14] [Interactiv]. Available: <https://livecharts.dev/>.
- [15] [Interactiv]. Available: https://en.wikipedia.org/wiki/Visual_Studio.
- [16] [Interactiv]. Available: <https://en.wikipedia.org/wiki/SQLite>.
- [17] [Interactiv]. Available: <https://en.wikipedia.org/wiki/Roboflow>.
- [18] [Interactiv]. Available: <https://en.wikipedia.org/wiki/Model%E2%80%93view%E2%80%93viewmodel>.
- [19] „Medii si Tehnologii de Programare,” [Interactiv]. Available: cv.upt.ro. [Accesat 2023 04 2023].
- [20] „w3schools,” [Interactiv]. Available: <https://www.w3schools.com/>. [Accesat 24 05 2023].
- [21] „freecodecamp,” [Interactiv]. Available: <https://www.freecodecamp.org/news/http-and-everything-you-need-to-know-about-it/>. [Accesat 06 05 2023].

DECLARAȚIE PRIVIND UTILIZAREA INTELIGENȚEI ARTIFICIALE

În cadrul realizării prezentei lucrări de licență au fost utilizate instrumente bazate pe inteligență artificială exclusiv cu rol de asistență în procesul de dezvoltare a aplicației și de redactare a documentației. Utilizarea acestora a avut scopul de a oferi explicații tehnice, clarificarea unor concepte și sugestii privind implementarea anumitor funcționalități, fără a înlocui contribuția proprie la realizarea proiectului.

În etapa de dezvoltare a scriptului Python utilizat pentru analiza exercițiilor fizice, inteligența artificială a fost folosită pentru înțelegerea și implementarea unor funcții matematice necesare procesării imaginilor, detectării obiectelor și organizării logicii aplicației. De asemenea, a fost utilizată pentru clarificarea utilizării unor biblioteci, identificarea și rezolvarea unor probleme de implementare, precum și pentru optimizarea unor secvențe de cod.

În dezvoltarea Web API-ului și a aplicației WPF, inteligența artificială a fost utilizată pentru obținerea unor explicații privind implementarea anumitor funcționalități, comunicarea dintre componentele aplicației și soluționarea unor erori întâlnite pe parcursul dezvoltării. Totodată, aceasta a fost folosită pentru îmbunătățirea exprimării academice, reformularea unor paragrafe și corectarea gramaticală și stilistică a documentației lucrării.

Toate informațiile și sugestiile generate au fost analizate, verificate și adaptate înainte de utilizare, iar implementarea finală a fost realizată și validată de către autor. Proiectarea aplicației, implementarea funcționalităților, integrarea componentelor software, testarea și redactarea formei finale a lucrării reprezintă contribuția proprie a autorului.

DECLARAȚIE DE AUTENTICITATE A
LUCRĂRII DE FINALIZARE A STUDIILOR*

Subsemnatul KOC SIS DENIS - ȘTEFAN

legitimat cu CI seria T3 nr. 760436

CNP 6040830350024

autorul lucrării DEZVOLTAREA UNEI APLICATII PENTRU MANAGEMENTUL UNEI SALI DE BOX

elaborată în vederea susținerii examenului de finalizare a studiilor de LICENȚA organizat de către Facultatea AUTOMATICĂ ȘI CALCULATOARE din cadrul Universității

Politehnica Timișoara, sesiunea IULIE 2026 a anului universitar 2025-2026, coordonator CONF. DR. ING. OCTAVIAN ȘTEFAN luând în

considerare prevederile Capitolului V - Măsuri privind asigurarea originalității lucrărilor din *Regulamentul privind organizarea și desfășurarea examenelor de licență/diplomă și disertație în Universitatea Politehnica Timișoara*, aprobat prin HS nr. 12/01.2026 și cunoscând faptul că în cazul constatării ulterioare a unor încălcări ale normelor de etică/a faptului că diploma a fost obținută prin mijloace frauduloase, voi suporta sancțiunile legale prevăzute de Legea nr. 199/2023 – Legea Învățământului Superior, declar pe proprie răspundere, că:

- această lucrare este rezultatul propriei activități intelectuale;
- lucrarea nu conține texte, date sau elemente de grafică din alte lucrări sau din alte surse fără ca acestea să nu fie citate, inclusiv situația în care sursa o reprezintă o altă lucrare/alte lucrări ale subsemnatului;
- sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor;
- această lucrare nu a mai fost prezentată în fața unei alte comisii de examen/prezentată public/publicată de licență/diplomă/disertație;
- În elaborarea lucrării am utilizat instrumente specifice inteligenței artificiale (IA) și anume CHATGPT (denumirea) _____ (sursa), pe care le-am citat în conținutul lucrării/nu am utilizat instrumente specifice inteligenței artificiale (IA)¹.

Declar că sunt de acord ca lucrarea să fie verificată prin orice modalitate legală pentru confirmarea originalității, consimțind inclusiv la introducerea conținutului său într-o bază de date în acest scop.

Timișoara,

Data

01/07/2026

Semnătura

[Signature]

*Declarația se completează de student, se semnează olograf de acesta și se inserează în lucrarea de finalizare a studiilor, la sfârșitul lucrării, ca parte integrantă.

¹ Se va păstra una dintre variante: 1 - s-a utilizat IA și se menționează sursa 2 – nu s-a utilizat IA